How ‘exact?‘ Works

Discrimination Trees and SolveByElim in Lean 4

Evgenia Karunus

August 3, 2026

Demo

Exact?

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ : l₃ = l₂) :
l₁.length ≤ l₃.length := by
  exact?
```

Exact?

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ : l₃ = l₂) :
l₁.length ≤ l₃.length := by
  exact le_of_le_of_eq h₁ (congrArg List.length (id (Eq.symm h₂)))
```

Exact?

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ : l₃ = l₂) :
l₁.length ≤ l₃.length := by
  exact le_of_le_of_eq h₁ (congrArg List.length (id (Eq.symm h₂)))
```

`Exact?` works by

- finding ONE lemma from the list of ALL lemmas ever (uses `DiscrTree`)
- dealing with the remaining goals using LOCAL hypotheses (uses `SolveByElim`)

Exact?

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ : l₃ = l₂) :
l₁.length ≤ l₃.length := by
  exact le_of_le_of_eq h₁ (congrArg List.length (id (Eq.symm h₂)))
```

`Exact?` works by

- finding ONE lemma from the list of ALL lemmas ever (uses `DiscrTree`)
- dealing with the remaining goals using LOCAL hypotheses (uses `SolveByElim`)

```
exact le_of_le_of_eq h₁ (congrArg List.length (id (Eq.symm h₂)))
```

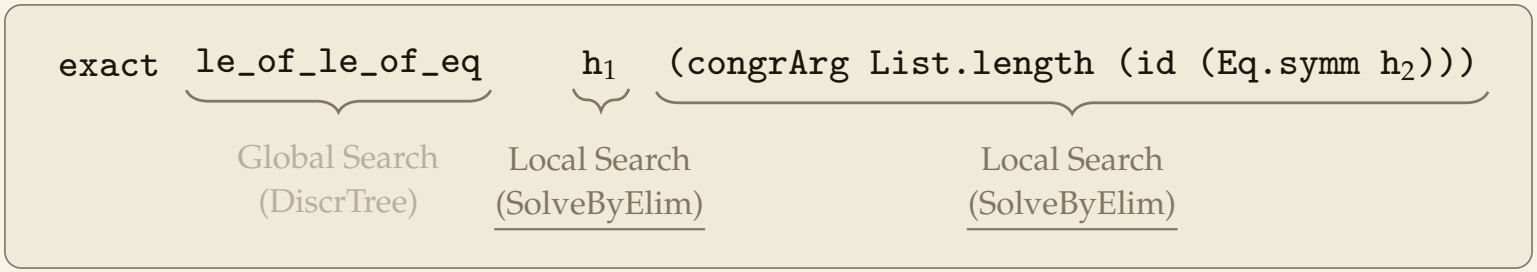
Global Search (DiscrTree) — `le_of_le_of_eq`

Local Search (SolveByElim) — `h₁`

Local Search (SolveByElim) — `(congrArg List.length (id (Eq.symm h₂)))`

Option	**Effect**
`exact? +grind/+try?`	use `grind` / `try?` as fallback discharger for subgoals
`exact? using h₁, h₂`	"provides identifiers in the local context that must be used"

Local Search



SolveByElim

> The `solve_by_elim` tactic has been ported from *Std* [aka Batteries] to *Lean* so that library search can use it.
>
> Lean v4.7.0 release notes

SolveByElim

> The `solve_by_elim` tactic has been ported from *Std* [aka Batteries] to *Lean* so that library search can use it.
>
> Lean v4.7.0 release notes

`solve_by_elim` as a tactic:

- Mathlib: 49 times
- Lean core: 8 times
- Batteries: 2 times

SolveByElim

> The `solve_by_elim` tactic has been ported from *Std* [aka Batteries] to *Lean* so that library search can use it.
>
> Lean v4.7.0 release notes

`solve_by_elim` as a tactic:

- Mathlib: 49 times
- Lean core: 8 times
- Batteries: 2 times

`solve_by_elim` (and `try?`) within tactic implementations:

- Mathlib: 10 tactics

  `tauto`, `mono`, `interval_cases`, `linarith`, `linear_combination`, `positivity`
- Lean core: 3 tactics

  `apply?`, `exact?`, `rw?`
- Batteries: 0 tactics

SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list: [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

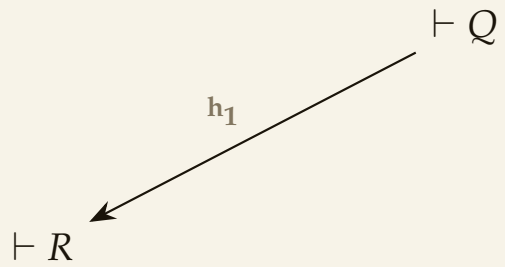
```
goal: ⊢ Q
```

SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list: [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal: ⊢ Q
```

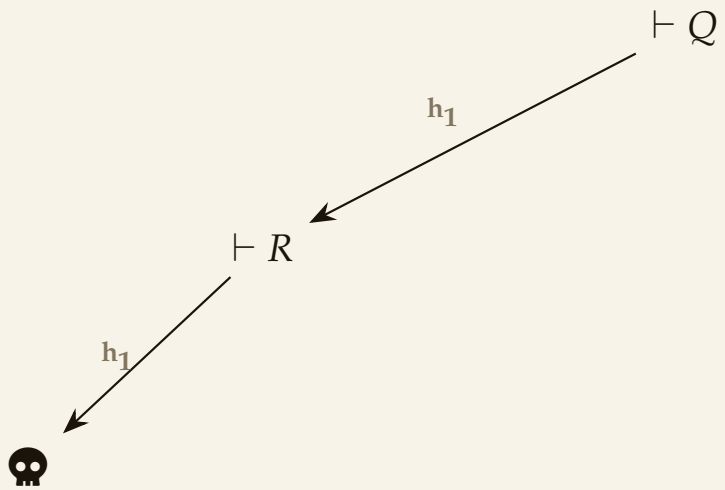


SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list: [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal: ⊢ Q
```

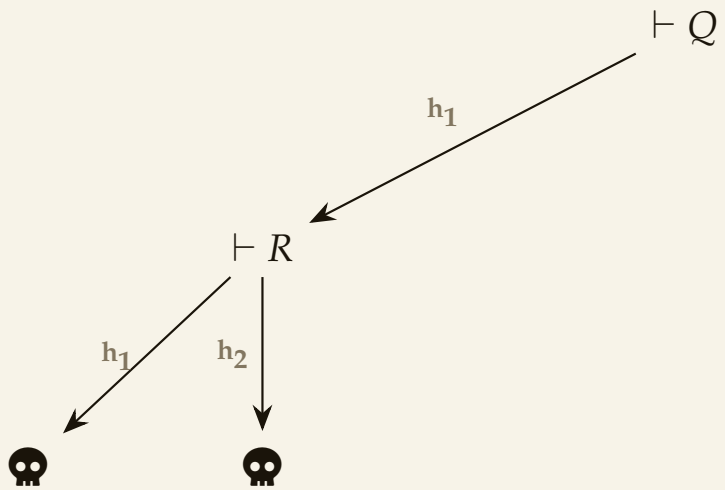


SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list: [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal: ⊢ Q
```

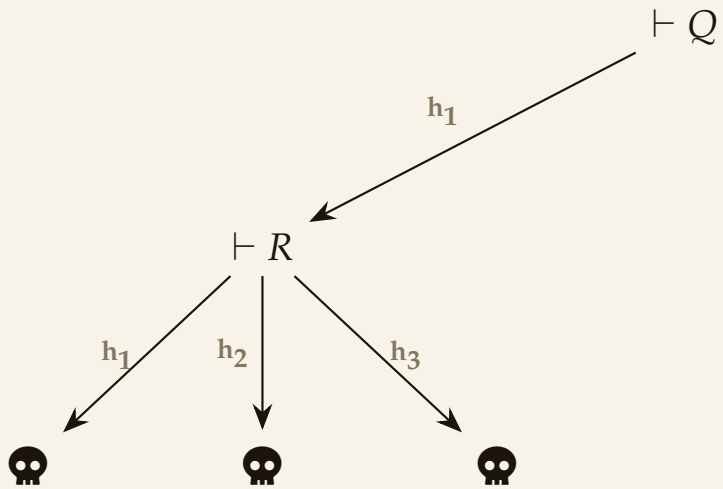


SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list: [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal: ⊢ Q
```

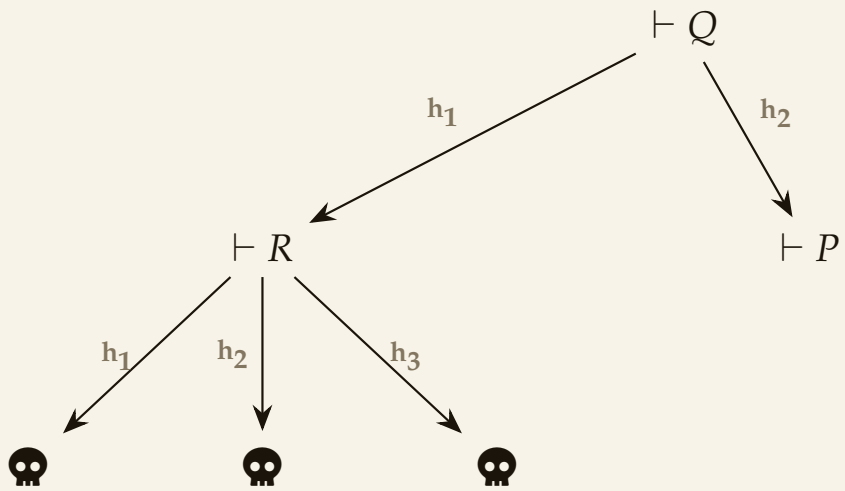


SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list: [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal: ⊢ Q
```

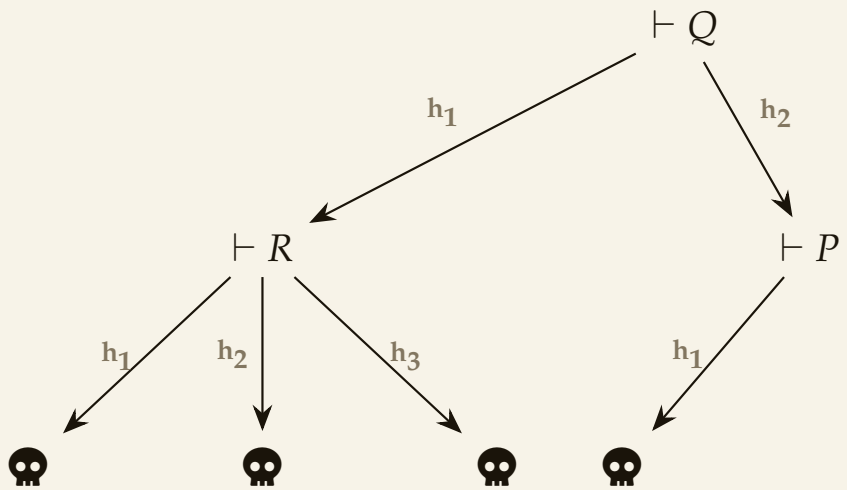


SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list:  [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal:  ⊢ Q
```

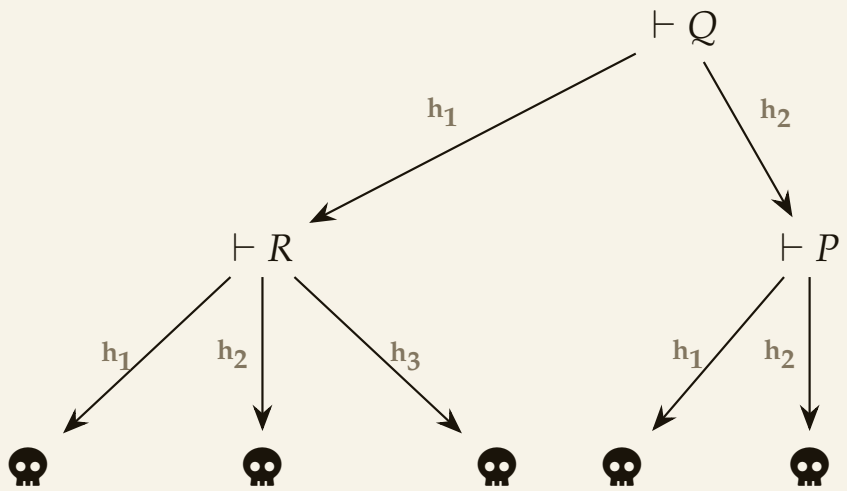


SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list:  [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal:  ⊢ Q
```

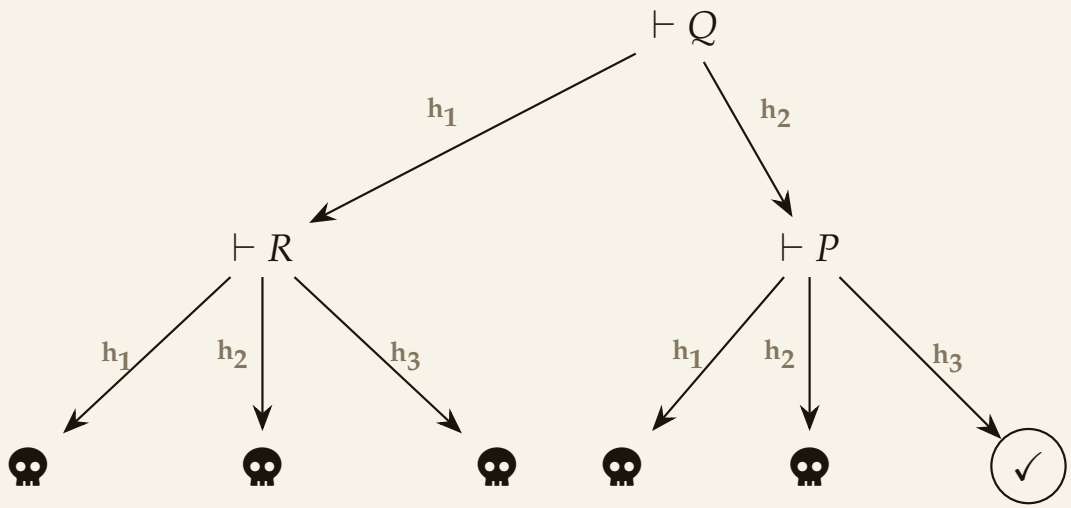


SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list:  [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal:  ⊢ Q
```

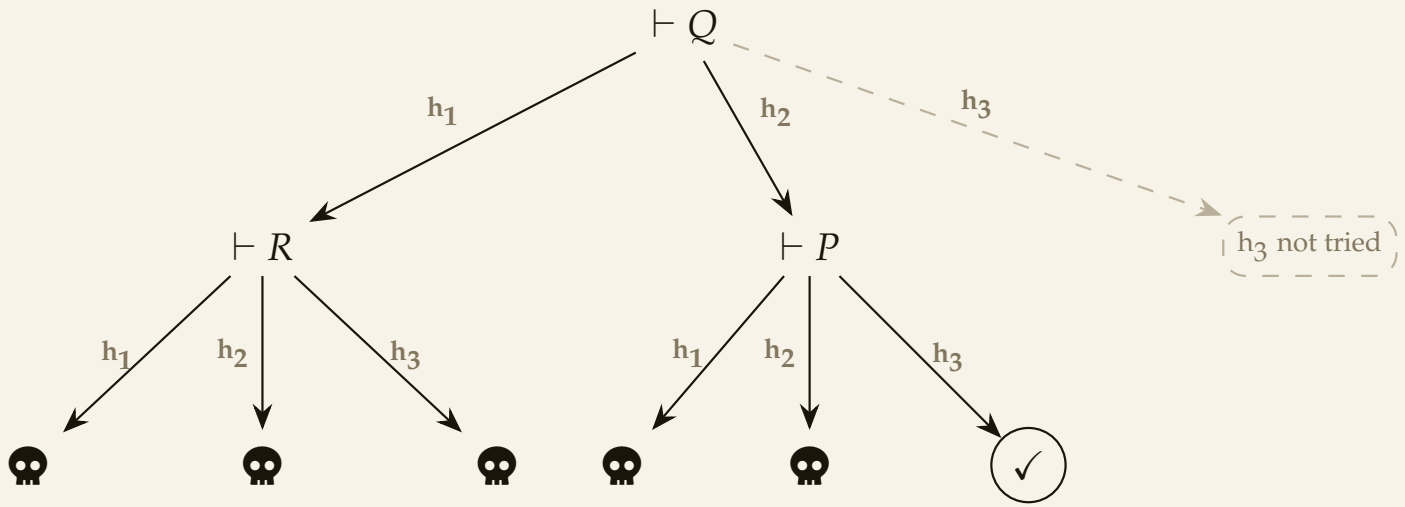


SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list:  [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal:  ⊢ Q
```

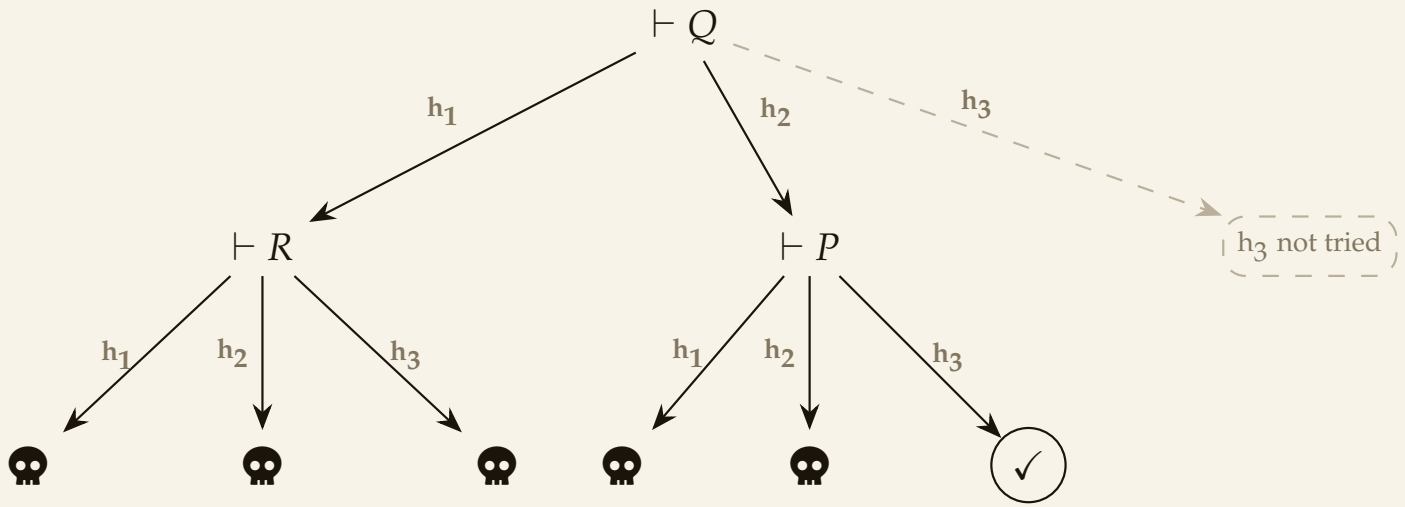


SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list:  [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal:  ⊢ Q
```



Maximum backtracking depth: 6.

Only successful application consumes depth coins.

Will find : `h₅ (h₄ (h₃ (h₂ h₁)))`

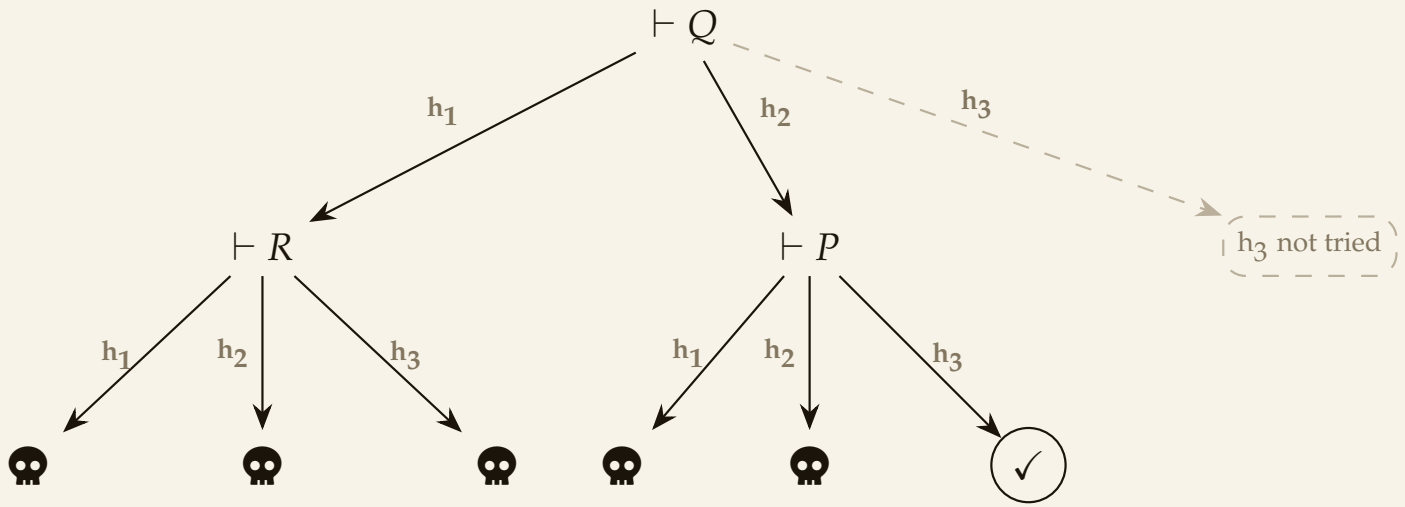
Won't find : `h₆ (h₅ (h₄ (h₃ (h₂ h₁))))`

SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list:  [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal:  ⊢ Q
```



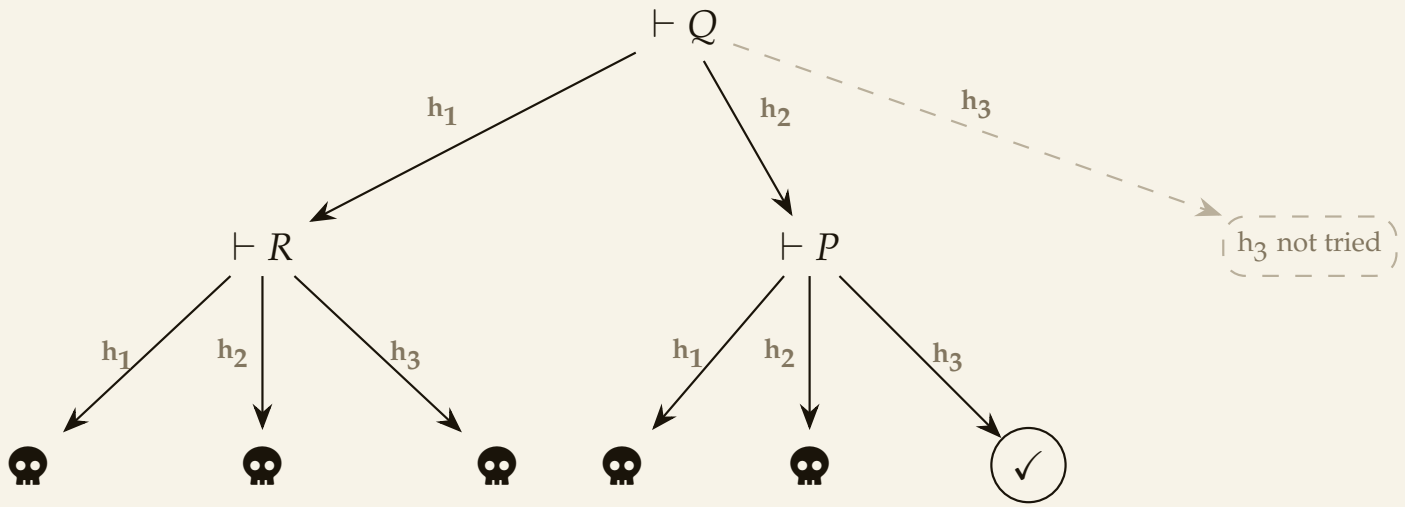
Try at each goal: `[h₁, h₂, h₃]`

SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list: [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal: ⊢ Q
```



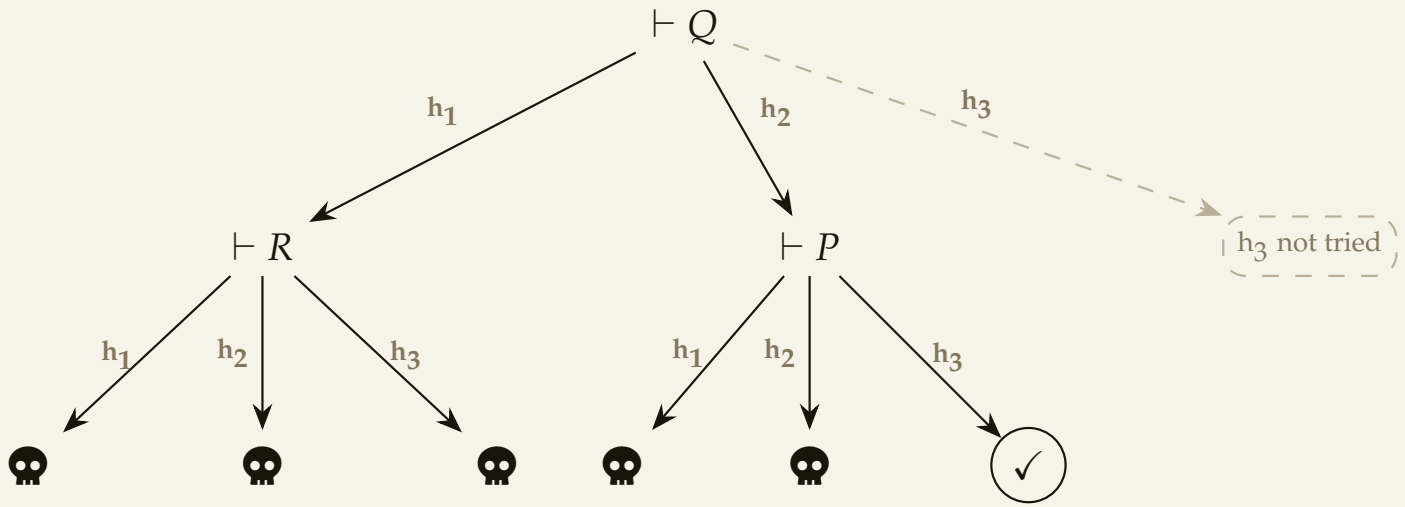
Try at each goal: `[h₁, h₂, h₃, constructor, intros, cfg.discharge]`

SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list:  [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal:  ⊢ Q
```



Try at each goal: `[h₁, h₂, h₃, ~~constructor~~, intros, cfg.discharge]`

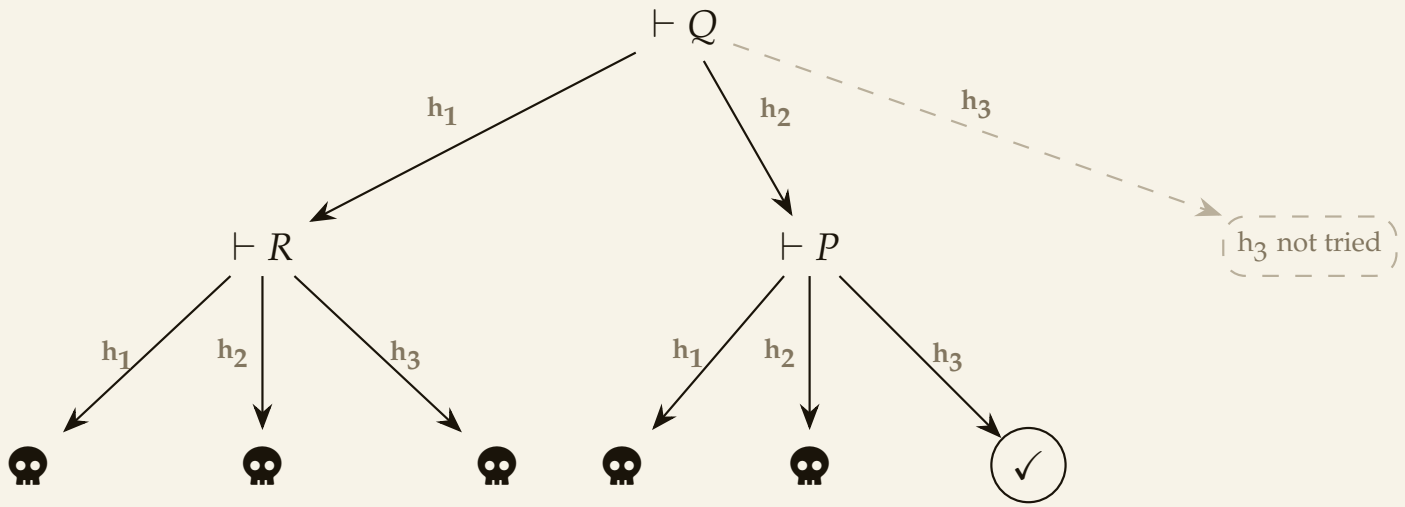
> **constructor** has been observed to significantly slow down **exact?** in Mathlib.
>
> */Meta/Tactic/LibrarySearch.lean* (Lean v4.29.0 source code)

SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list:  [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal:  ⊢ Q
```



Try at each goal: `[h₁, h₂, h₃, ~~constructor~~, intros, cfg.discharge]`

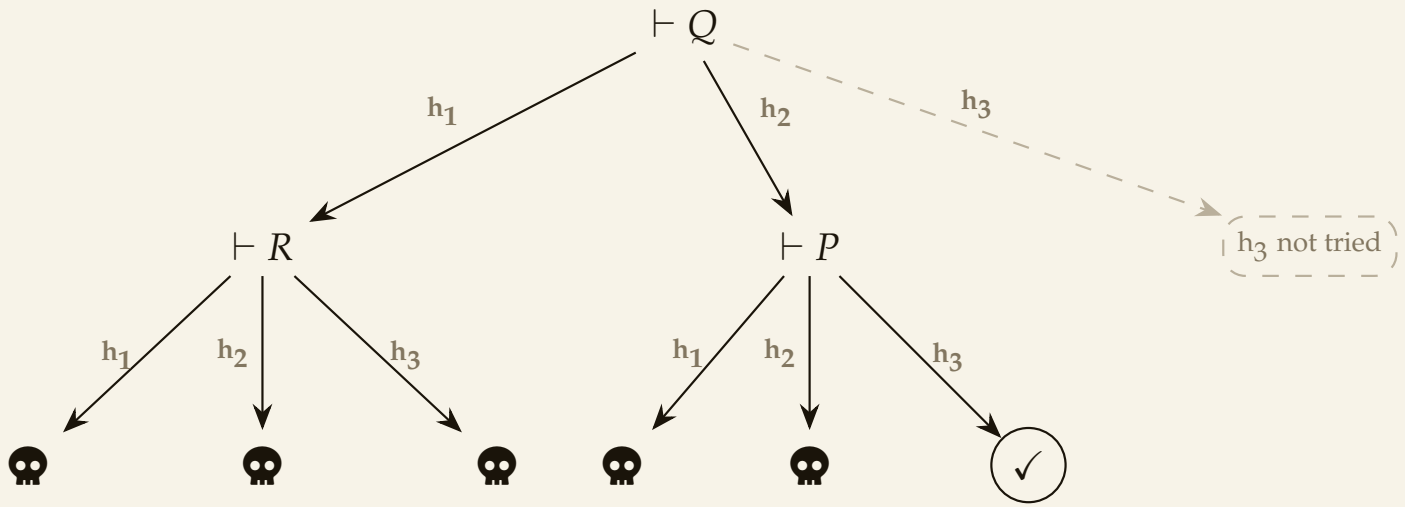
```
exact?  +grind
exact?  +try?
```

SolveByElim: Implementation

`SolveByElim` "eliminates" the current goal by running [apply, apply, apply...]

```
lemma list:  [h₁ : R → Q, h₂ : P → Q, h₃ : P]
```

```
goal:  ⊢ Q
```



Try at each goal: `[lemma_list, constructor, intros, cfg.discharge]`

- `[lemma_list, ...]` - normal backtracking
- `[..., constructor, intros, ...]` - doesn't try intros&discharger if child fails
- `[..., cfg.discharge]` - just exits the tree on fail

SolveByElim: Lemma List

Default `solve_by_elim` lemma list:

- **4 hardcoded functions**

```
[
  rfl : a = a,
  trivial : True,
  congrArg : (a = b) (f : α → β) → f a = f b,
  congrFun : (f = g) (a : α) → f a = g a
]
```

- **local hypotheses AND their symmetric versions**

```
[
   ...all hypotheses above the ⊢ in the tactic state,
   ...their symmetric versions
]
```

SolveByElim: Lemma List

Options of the `solve_by_elim` tactic:

Option Used	Lemma List
`solve_by_elim [t₁, t₂]`	**default lemma list +** `[t₁, t₂]`
`solve_by_elim only [t₁, t₂]`	`[t₁, t₂]`
`solve_by_elim using [a₁, a₂]`	**default lemma list +** all lemmas with attrs `[a₁, a₂]`

Options of the `exact?` tactic:

Option Used	Lemma List
`exact?  using [h₁, h₂]`	**FILTER** the solutions by the presence of `[h₁, h₂]`

SolveByElim: Real Example

```
set_option trace.Meta.Tactic.solveByElim true in
example (f :  Nat → Nat) (g :  Nat → Nat) (h :  g = f) :  f 42 = g 42 := by
  solve_by_elim
```

SolveByElim: Real Example

```
set_option trace.Meta.Tactic.solveByElim true in
example (f : Nat → Nat) (g : Nat → Nat) (h : g = f) : f 42 = g 42 := by
  solve_by_elim
```

```
lemma list: [
  rfl : a = a,
  trivial : True,
  congrArg : (a = b) (f : α → β) → f a = f b,
  congrFun : (f = g) (a : α) → f a = g a,
  f : Nat → Nat,
  g : Nat → Nat,
  h : g = f,
  id (Eq.symm h) : f = g
]
```

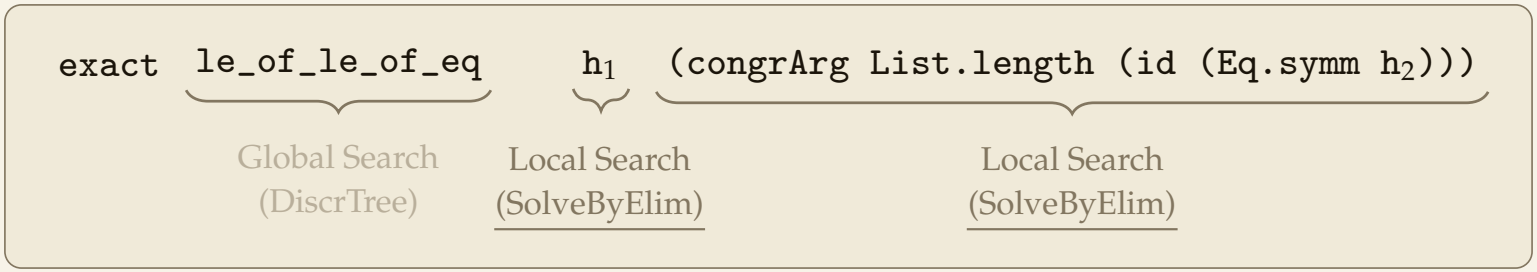
SolveByElim: Real Example

```
set_option trace.Meta.Tactic.solveByElim true in
example (f :  Nat → Nat) (g :  Nat → Nat) (h :  g = f) :  f 42 = g 42 := by
  solve_by_elim
```

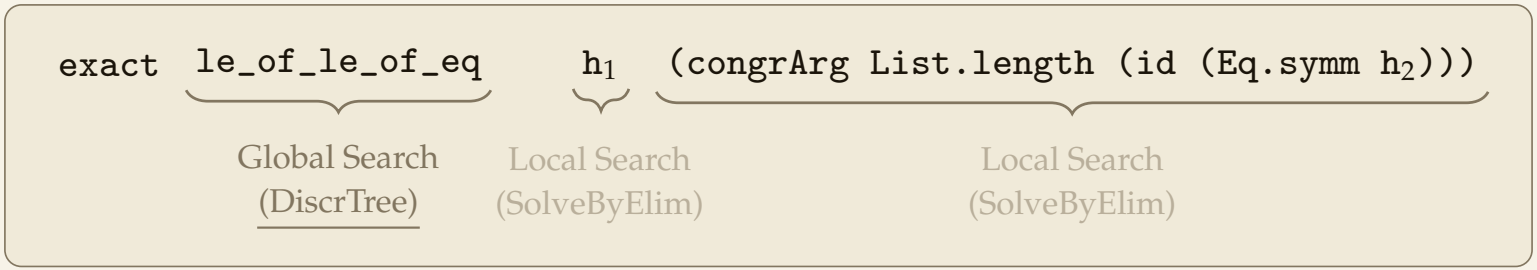
working on:
f g : Nat → Nat
h : g = f
⊢ f 42 = g 42

- ✗ trying to apply: f
- ✗ trying to apply: g
- ✗ trying to apply: h
- ✗ trying to apply: id (Eq.symm h)
- ✗ trying to apply: rfl
- ✗ trying to apply: trivial
- ✓ trying to apply: congrFun
- working on:
  f g : Nat → Nat
  h : g = f
  ⊢ f = g
  - ✗ trying to apply: f
  - ✗ trying to apply: g
  - ✗ trying to apply: h
  - ✓ trying to apply: id (Eq.symm h)
  - ✓ success: exact congrFun (id (Eq.symm h)) 42

Local Search



Global Search



Global Search

```
exact le_of_le_of_eq h₁ (congrArg List.length (id (Eq.symm h₂)))
```

le_of_le_of_eq: Global Search (DiscrTree)
h₁: Local Search (SolveByElim)
(congrArg List.length (id (Eq.symm h₂))): Local Search (SolveByElim)

What we're proving:

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ : l₃ = l₂) :
l₁.length ≤ l₃.length := by
  exact?
```

Our mathlib:

```
add_2_2                                    : 2 + 2 = 4
add_2_666                                  : 2 + 666 = 668
add_3_5                                    : 3 + 5 = 8
le_of_le_of_eq (h₁ : a ≤ b) (h₂ : b = c) : a ≤ c
add_3_7                                    : 3 + 7 = 10
```

Global Search: Linear Search

```
exact le_of_le_of_eq h₁ (congrArg List.length (id (Eq.symm h₂)))
```

Global Search (DiscrTree) | Local Search (SolveByElim) | Local Search (SolveByElim)

What we're proving:

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ : l₃ = l₂) :
l₁.length ≤ l₃.length := by
  exact?
```

Our mathlib:

```
add_2_2                                    : 2 + 2 = 4
add_2_666                                  : 2 + 666 = 668
add_3_5                                    : 3 + 5 = 8
le_of_le_of_eq (h₁ : a ≤ b) (h₂ : b = c) : a ≤ c
add_3_7                                    : 3 + 7 = 10
```

Global Search: Linear Search

```
exact le_of_le_of_eq h₁ (congrArg List.length (id (Eq.symm h₂)))
```

le_of_le_of_eq: Global Search (DiscrTree)
h₁: Local Search (SolveByElim)
(congrArg List.length (id (Eq.symm h₂))): Local Search (SolveByElim)

What we're proving:

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ : l₃ = l₂) :
l₁.length ≤ l₃.length := by
  exact?
```

Our mathlib:

```
add_2_2                                    : 2 + 2 = 4
add_2_666                                  : 2 + 666 = 668
add_3_5                                    : 3 + 5 = 8
le_of_le_of_eq (h₁ : a ≤ b) (h₂ : b = c) : a ≤ c
add_3_7                                    : 3 + 7 = 10
```

Global Search: Linear Search

```
exact le_of_le_of_eq h₁ (congrArg List.length (id (Eq.symm h₂)))
```

le_of_le_of_eq: Global Search (DiscrTree)
h₁: Local Search (SolveByElim)
(congrArg List.length (id (Eq.symm h₂))): Local Search (SolveByElim)

What we're proving:

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ : l₃ = l₂) :
l₁.length ≤ l₃.length := by
  exact?
```

Our mathlib:

```
add_2_2                                    : 2 + 2 = 4
add_2_666                                  : 2 + 666 = 668
add_3_5                                    : 3 + 5 = 8
le_of_le_of_eq (h₁ : a ≤ b) (h₂ : b = c) : a ≤ c
add_3_7                                    : 3 + 7 = 10
```

Global Search: Linear Search

```
exact le_of_le_of_eq h₁ (congrArg List.length (id (Eq.symm h₂)))
```

le_of_le_of_eq: Global Search (DiscrTree)
h₁: Local Search (SolveByElim)
(congrArg List.length (id (Eq.symm h₂))): Local Search (SolveByElim)

What we're proving:

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ : l₃ = l₂) :
l₁.length ≤ l₃.length := by
  exact?
```

Our mathlib:

```
add_2_2                                    : 2 + 2 = 4
add_2_666                                  : 2 + 666 = 668
add_3_5                                    : 3 + 5 = 8
le_of_le_of_eq (h₁ : a ≤ b) (h₂ : b = c) : a ≤ c
add_3_7                                    : 3 + 7 = 10
```

Global Search: Linear Search

What we're proving:

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ : l₃ = l₂) :
l₁.length ≤ l₃.length := by
  exact?
```

Our mathlib:

```
add_2_2                                    : 2 + 2 = 4
add_2_666                                  : 2 + 666 = 668
add_3_5                                    : 3 + 5 = 8
le_of_le_of_eq (h₁ : a ≤ b) (h₂ : b = c) : a ≤ c
add_3_7                                    : 3 + 7 = 10
```

Fun fact - in Lean 3, `exact?` used linear search!

" `library_search` is just a linear search over the entire environment. We do a tiny bit of filtering, but in the end `apply` is so incredibly fast (in particular, it fails fast) that mostly it is better to just hand everything over to it. I was shocked when I first tried it out that an approach as dumb as that was actually viable."

Kim Morrison, 2021

Indexing Theorems

Path Indexing

Discrimination Trees

> Term indexing was used in the early days of automated deduction, but it was undocumented or not emphasized in the literature, and the methods are not well known. As a result, it appears that several of the methods were 'reinvented'. The origins of the two indexing methods remain uncertain. The roots of discrimination-tree indexing appear to lie directly in formula manipulation systems, and the roots of path indexing appear to lie in database technology.
>
> "Experiments with Discrimination-Tree Indexing and Path Indexing"
> William McCune, 1990

Path Indexing

Our mathlib:

Theorem Name	Statement	Prefix Form
add_2_2	$2+2=4$	$=(+(2,\ 2),\ 4)$
add_2_666	$2+666=668$	$=(+(2,\ 666),\ 668)$
add_3_5	$3+5=8$	$=(+(3,\ 5),\ 8)$
add_3_7	$3+7=10$	$=(+(3,\ 7),\ 10)$
le_of_le_of_eq	$a \leq c$	$\leq(*,\ *)$

Path Indexing

Our mathlib:

Theorem Name	Statement	Prefix Form
add_2_2	$2+2=4$	$=(+(2,\ 2),\ 4)$
add_2_666	$2+666=668$	$=(+(2,\ 666),\ 668)$
add_3_5	$3+5=8$	$=(+(3,\ 5),\ 8)$
add_3_7	$3+7=10$	$=(+(3,\ 7),\ 10)$
le_of_le_of_eq	$a \leq c$	$\leq(*,\ *)$

Each theorem generates a number of "paths".

Each "path" has the form `[symbol, arg-position, symbol, arg-position, ..., symbol]`.

For example, `add_3_7: = (+ (3, 7), 10)` has five symbols, so it generates five paths:

Symbols (add_3_7)	Path
$=$	$[=]$
$+$	$[=,\ 1,\ +]$
3	$[=,\ 1,\ +,\ 1,\ 3]$
7	$[=,\ 1,\ +,\ 2,\ 7]$
10	$[=,\ 2,\ 10]$

Per each theorem, per each symbol: compute its path:

Symbols (add_3_7)	Path
=	[=]
+	[=, 1, +]
3	[=, 1, +, 1, 3]
7	[=, 1, +, 2, 7]
10	[=, 2, 10]

Symbols (add_3_5)	Path
=	[=]
+	[=, 1, +]
3	[=, 1, +, 1, 3]
5	[=, 1, +, 2, 5]
8	[=, 2, 8]

And put those paths into a new table:

Path	add_3_5	add_3_7
[=]	✓	✓
[=, 1, +]	✓	✓
[=, 1, +, 1, 3]	✓	✓
[=, 1, +, 2, 5]	✓	
[=, 1, +, 2, 7]		✓
[=, 2, 8]	✓	
[=, 2, 10]		✓

Path Indexing

Path	add_3_5	add_3_7
`[=]`	✓	✓
`[=, 1, +]`	✓	✓
`[=, 1, +, 1, 3]`	✓	✓
`[=, 1, +, 2, 5]`	✓	
`[=, 1, +, 2, 7]`		✓
`[=, 2, 8]`	✓	
`[=, 2, 10]`		✓

Path Indexing

Path	add_3_5	add_3_7
`[=]`	✓	✓
`[=, 1, +]`	✓	✓
`[=, 1, +, 1, 3]`	✓	✓
`[=, 1, +, 2, 5]`	✓	
`[=, 1, +, 2, 7]`		✓
`[=, 2, 8]`	✓	
`[=, 2, 10]`		✓

Suppose our goal is `3 + 7 = 10`.

Compute the path of each symbol of our goal, and intersect the theorem sets:

`[=]`	$\{\textbf{add\_3\_5}, \textbf{add\_3\_7}\}$
`[=, 1, +]`	$\cap \{\textbf{add\_3\_5}, \textbf{add\_3\_7}\}$
`[=, 1, +, 1, 3]`	$\cap \{\textbf{add\_3\_5}, \textbf{add\_3\_7}\}$
`[=, 1, +, 2, 7]`	$\cap \{\textbf{add\_3\_7}\}$
`[=, 2, 10]`	$\cap \{\textbf{add\_3\_7}\}$
	$= \{\textbf{add\_3\_7}\}$

We found our theorem candidate: add_3_7.

Indexing Theorems

Path Indexing

Discrimination Trees

> Two different indexing methods are used: discrimination-tree indexing and path indexing. Some types of term are much better handled by one or the other of the indexing methods. Thus, no clear winner existed between the two methods.
> ...
> The strongest conclusion from the experiments is that discrimination indexing is a clear winner over path indexing for generalization retrieval on the types of term with which I experimented.
>
> "Experiments with Discrimination-Tree Indexing and Path Indexing"
> William McCune, 1990

Indexing Theorems

Path Indexing

- Z3

Discrimination Trees

- Lean ("Discrimination Tree")
- Rocq ("Discrimination Net")
- Isabelle ("Discrimination Net")
- E Prover (Fingerprint Indexing variant)
- Vampire (Substitution Tree variant)

> Two different indexing methods are used: discrimination-tree indexing and path indexing. Some types of term are much better handled by one or the other of the indexing methods. Thus, no clear winner existed between the two methods.
>
> ...
>
> The strongest conclusion from the experiments is that discrimination indexing is a clear winner over path indexing for generalization retrieval on the types of term with which I experimented.
>
> "Experiments with Discrimination-Tree Indexing and Path Indexing"
> William McCune, 1990

Discrimination Trees

Discrimination tree is a trie

Discrimination Trees

Discrimination tree is a trie

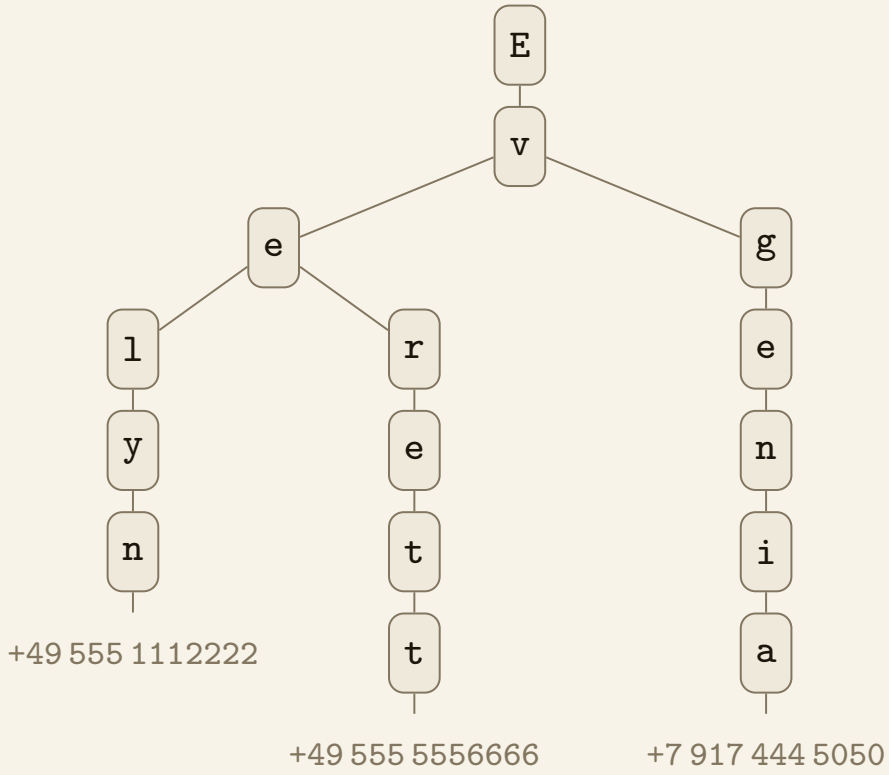
Trie nodes is something we do know
Trie leaves is something we WANT to know

Discrimination Trees

Discrimination tree is a trie

Trie nodes is something we do know
Trie leaves is something we WANT to know

For example,



Discrimination Trees

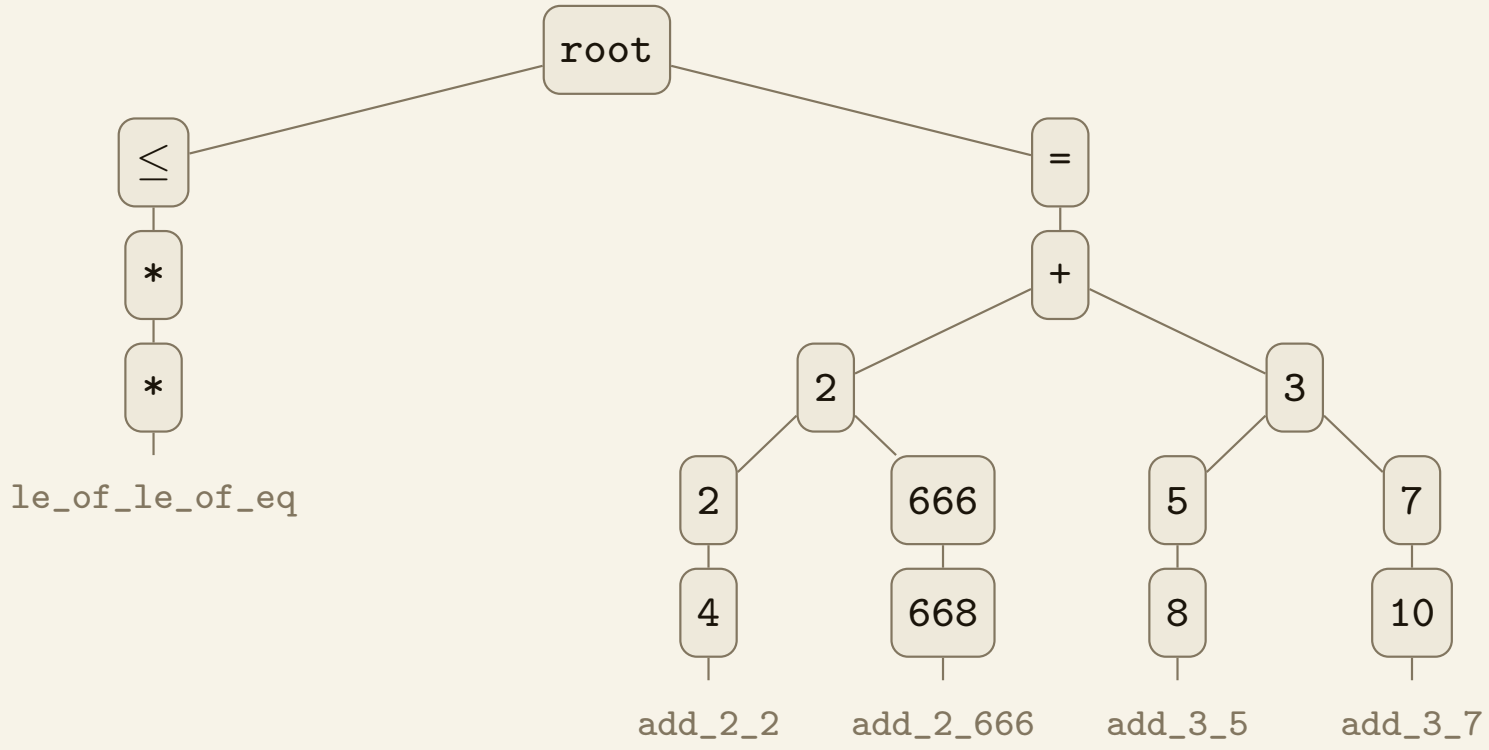
Our mathlib:

Theorem Name	Statement	Prefix Form
add_2_2	$2+2=4$	$=(+(2,\ 2),\ 4)$
add_2_666	$2+666=668$	$=(+(2,\ 666),\ 668)$
add_3_5	$3+5=8$	$=(+(3,\ 5),\ 8)$
add_3_7	$3+7=10$	$=(+(3,\ 7),\ 10)$
le_of_le_of_eq	$a \leq c$	$\leq(*,\ *)$

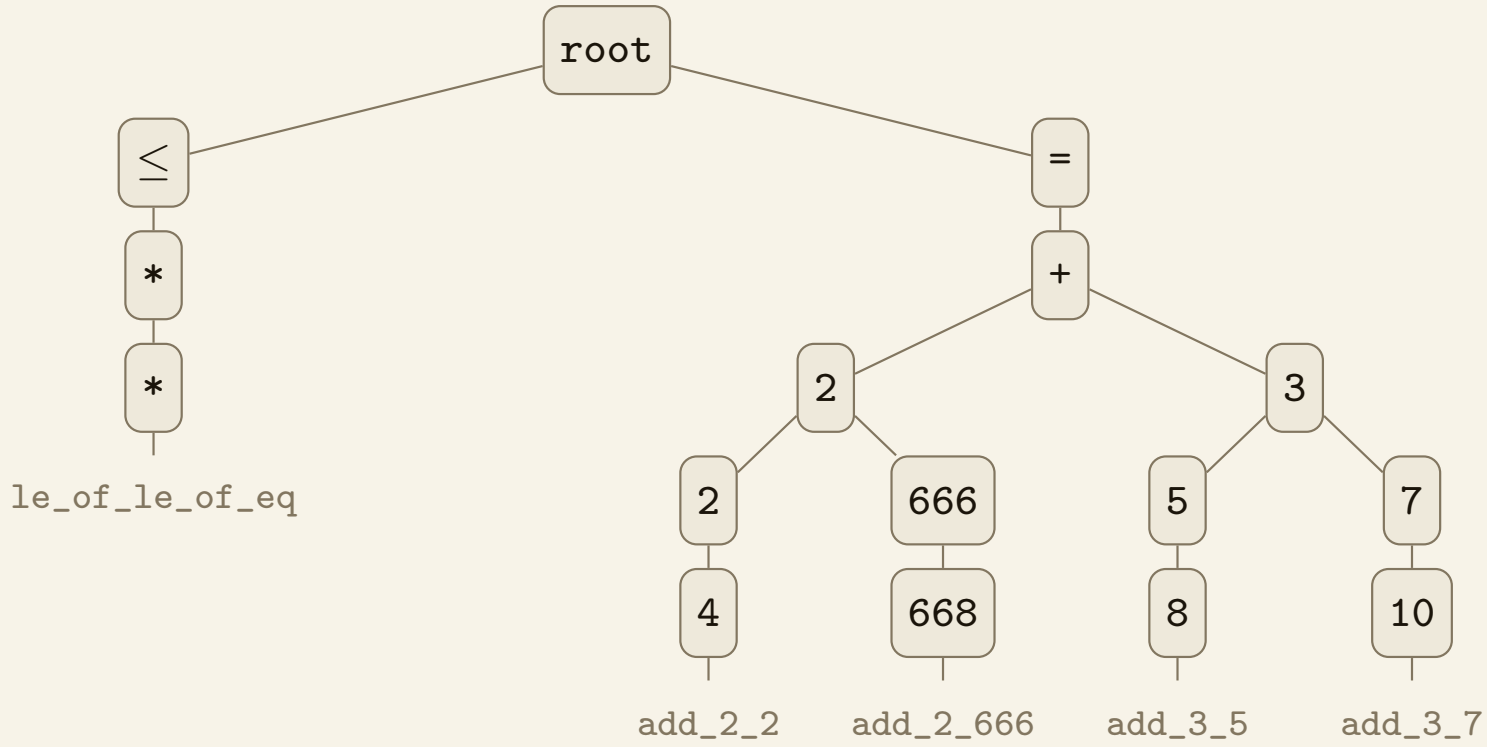
Discrimination Trees

Our mathlib:

Theorem Name	Statement	Prefix Form
add_2_2	$2+2=4$	$=(+(2,\,2),\,4)$
add_2_666	$2+666=668$	$=(+(2,\,666),\,668)$
add_3_5	$3+5=8$	$=(+(3,\,5),\,8)$
add_3_7	$3+7=10$	$=(+(3,\,7),\,10)$
le_of_le_of_eq	$a \le c$	$\le(*,\,*)$



Discrimination Trees



This is our discrimination tree, aka `DiscrTree`.

Lean and Mathlib maintain around 15 different `DiscrTree`s. Nodes always contain Lean expressions, leaves contain different stuff.

`DiscrTree α`: nodes are expressions, each leaf is α.
`DiscrTree Name`: nodes are expressions, each leaf is a `Name`
`DiscrTree SimpTheorem`: nodes are expressions, each leaf is a `SimpTheorem`

Discrimination Trees: Lean and Mathlib

Used By	What's Stored	DiscrTree α
typeclass instances	`@[instance]`	`DiscrTree InstanceEntry`
unification hints	`@[unification_hint]`	`DiscrTree Name`
`simp`	`@[simp]`	`DiscrTree SimpTheorem`
`simp` (simprocs)	`@[simproc]`	`DiscrTree SimprocEntry`
`rfl`	`@[refl]`	`DiscrTree Name`
`symm`	`@[symm]`	`DiscrTree Name`
`ext`	`@[ext]`	`DiscrTree ExtTheorem`
`norm_num`	`@[norm_num]`	`DiscrTree NormNumExt`
`positivity`	`@[positivity]`	`DiscrTree PositivityExt`
`push`	`@[push]`	`DiscrTree SimpTheorem`
`pull`	`@[push]`	`DiscrTree PullTheorem`
`fun_prop`	`@[fun_prop]`	`RefinedDiscrTree GeneralTheorem`
`exact?` / `apply?`	all imported declarations	`LazyDiscrTree (Name × DeclMod)`
`rw?`	all imported declarations	`LazyDiscrTree (Name × RwDirection)`
`rw??`	all imported declarations	`RefinedDiscrTree RewriteLemma`

Discrimination Trees: Lean and Mathlib

Used By	**What's Stored**	**DiscrTree α**
typeclass instances	`@[instance]`	`DiscrTree InstanceEntry`
unification hints	`@[unification_hint]`	`DiscrTree Name`
`simp`	`@[simp]`	`DiscrTree SimpTheorem`
	...	
`exact?` / `apply?`	all imported declarations	`LazyDiscrTree (Name × DeclMod)`
`rw??`	all imported declarations	`RefinedDiscrTree RewriteLemma`

> During the compilation of Mathlib, Lean spends approximately 14% of its time running the discrimination trees that allow for efficient association of terms with values.
>
> "Growing Mathlib: maintenance of a large scale mathematical library"
> Baanen, Ballard, Commelin, Chen, Rothgang, Testa, 2025

Discrimination Trees: Lean and Mathlib

Used By	**What's Stored**	**DiscrTree α**
typeclass instances	`@[instance]`	`DiscrTree InstanceEntry`
unification hints	`@[unification_hint]`	`DiscrTree Name`
`simp`	`@[simp]`	`DiscrTree SimpTheorem`
	...	
`exact?` / `apply?`	all imported declarations	`LazyDiscrTree (Name × DeclMod)`
`rw??`	all imported declarations	`RefinedDiscrTree RewriteLemma`

> During the compilation of Mathlib, Lean spends approximately 14% of its time running the discrimination trees that allow for efficient association of terms with values.
>
> "Growing Mathlib: maintenance of a large scale mathematical library"
> Baanen, Ballard, Commelin, Chen, Rothgang, Testa, 2025

A single large discrimination tree?

Discrimination Trees: Lean and Mathlib

Used By	**What's Stored**	**DiscrTree α**
typeclass instances	`@[instance]`	`DiscrTree InstanceEntry`
unification hints	`@[unification_hint]`	`DiscrTree Name`
`simp`	`@[simp]`	`DiscrTree SimpTheorem`
	...	
`exact?` / `apply?`	all imported declarations	`LazyDiscrTree (Name × DeclMod)`
`rw??`	all imported declarations	`RefinedDiscrTree RewriteLemma`

> During the compilation of Mathlib, Lean spends approximately 14% of its time running the discrimination trees that allow for efficient association of terms with values.
>
> "Growing Mathlib: maintenance of a large scale mathematical library"
> Baanen, Ballard, Commelin, Chen, Rothgang, Testa, 2025

A single large discrimination tree?

Discrimination Trees: Lean and Mathlib

Used By	**What's Stored**	**DiscrTree α**
typeclass instances	`@[instance]`	`DiscrTree InstanceEntry`
unification hints	`@[unification_hint]`	`DiscrTree Name`
`simp`	`@[simp]`	`DiscrTree SimpTheorem`
	...	
`exact?` / `apply?`	all imported declarations	`LazyDiscrTree (Name × DeclMod)`
`rw??`	all imported declarations	`RefinedDiscrTree RewriteLemma`

> During the compilation of Mathlib, Lean spends approximately 14% of its time running the discrimination trees that allow for efficient association of terms with values.
>
> "Growing Mathlib: maintenance of a large scale mathematical library"
> Baanen, Ballard, Commelin, Chen, Rothgang, Testa, 2025

A single large discrimination tree?

- Leaves store different stuff (e.g. `InstanceEntry` vs `Name`) - not a problem

Discrimination Trees: Lean and Mathlib

Used By	**What's Stored**	**DiscrTree α**
typeclass instances	`@[instance]`	`DiscrTree InstanceEntry`
unification hints	`@[unification_hint]`	`DiscrTree Name`
`simp`	`@[simp]`	`DiscrTree SimpTheorem`
	...	
`exact?` / `apply?`	all imported declarations	`LazyDiscrTree (Name × DeclMod)`
`rw??`	all imported declarations	`RefinedDiscrTree RewriteLemma`

> During the compilation of Mathlib, Lean spends approximately 14% of its time running the discrimination trees that allow for efficient association of terms with values.
>
> "Growing Mathlib: maintenance of a large scale mathematical library"
> Baanen, Ballard, Commelin, Chen, Rothgang, Testa, 2025

A single large discrimination tree?

- Leaves store different stuff (e.g. `InstanceEntry` vs `Name`) - not a problem
- Different theorems are processed (e.g. `@[simp]` vs `@[norm_num]`) - not a problem

Discrimination Trees: Lean and Mathlib

Used By	**What's Stored**	**DiscrTree α**
typeclass instances	`@[instance]`	`DiscrTree InstanceEntry`
unification hints	`@[unification_hint]`	`DiscrTree Name`
`simp`	`@[simp]`	`DiscrTree SimpTheorem`
	...	
`exact?` / `apply?`	all imported declarations	`LazyDiscrTree (Name × DeclMod)`
`rw??`	all imported declarations	`RefinedDiscrTree RewriteLemma`

> During the compilation of Mathlib, Lean spends approximately 14% of its time running the discrimination trees that allow for efficient association of terms with values.
>
> "Growing Mathlib: maintenance of a large scale mathematical library"
> Baanen, Ballard, Commelin, Chen, Rothgang, Testa, 2025

A single large discrimination tree?

- Leaves store different stuff (e.g. `InstanceEntry` vs `Name`) - not a problem
- Different theorems are processed (e.g. `@[simp]` vs `@[norm_num]`) - not a problem
- Actual reason: **nodes** store different stuff (e.g. `exact?` full expression, `simp` LHS)

Global Search: Example

Our Mathlib:

```
theorem Nat.add_comm : ∀ (n m : Nat), n + m = m + n
theorem Nat.zero_le  : ∀ (n : Nat), 0 ≤ n
theorem Nat.le_refl  : ∀ (n : Nat), n ≤ n
theorem Nat.le_succ  : ∀ (n : Nat), n ≤ n.succ
```

Our goal:

```
⊢ 0 ≤ 5
```

We call `exact?`, and the global search among all Mathlib lemmas starts:

- We convert each Mathlib theorem into `[Key, Key, Key]`, and put all theorems into a **DiscrTree**
- We convert our goal into `[Key, Key, Key]`
- We match the resulting **DiscrTree** to our goal!

Global Search: What We Do To Mathlib

```
theorem Nat.add_comm :  ∀ (n m :  Nat), n + m = m + n
```

Global Search: What We Do To Mathlib

```
theorem Nat.add_comm :  ∀ (n m :  Nat), n + m = m + n
theorem Nat.add_comm :  ∀ (n m :  Nat), Eq Nat (Nat.add n m) (Nat.add m n)
```

Global Search: What We Do To Mathlib

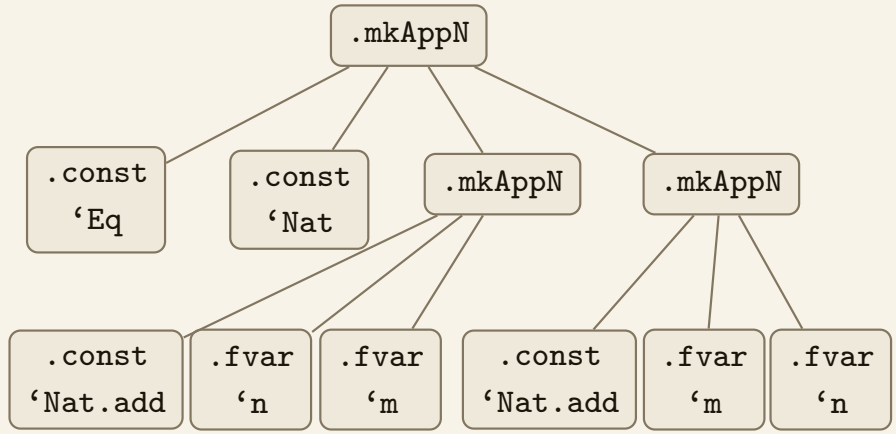
```
theorem Nat.add_comm :  ∀ (n m :  Nat), n + m = m + n
theorem Nat.add_comm :  ∀ (n m :  Nat), Eq Nat (Nat.add n m) (Nat.add m n)
theorem Nat.add_comm :  .mkAppN (.const `Eq) #[.const `Nat, .mkAppN (.const
`Nat.add) #[.fvar `n, .fvar `m], .mkAppN (.const `Nat.add) #[.fvar `m, .fvar
`n]]
```

```
inductive Expr
 | bvar    :  Nat
 | fvar    :  FVarId
 | mvar    :  MVarId
 | sort    :  Level
 | const   :  Name → List Level
 | app     :  Expr → Expr
 | lam     :  Name → Expr → Expr
 | forallE :  Name → Expr → Expr
 | letE    :  Name → ...
 | lit     :  Literal
 | mdata   :  MData → Expr
 | proj    :  Name → Nat → Expr
```

Global Search: What We Do To Mathlib

```
theorem Nat.add_comm : ∀ (n m : Nat), n + m = m + n
theorem Nat.add_comm : ∀ (n m : Nat), Eq Nat (Nat.add n m) (Nat.add m n)
theorem Nat.add_comm : .mkAppN (.const `Eq) #[.const `Nat, .mkAppN (.const
`Nat.add) #[.fvar `n, .fvar `m], .mkAppN (.const `Nat.add) #[.fvar `m, .fvar
`n]]
```

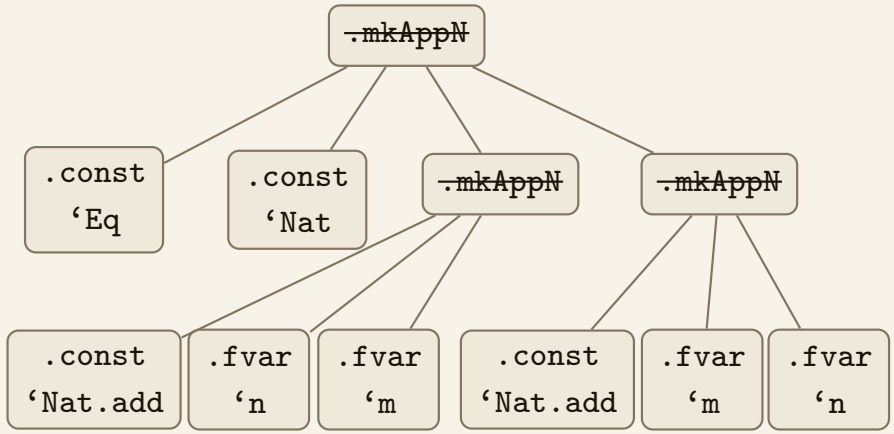
```
inductive Expr
 | bvar    :  Nat
 | fvar    :  FVarId
 | mvar    :  MVarId
 | sort    :  Level
 | const   :  Name → List Level
 | app     :  Expr → Expr
 | lam     :  Name → Expr → Expr
 | forallE :  Name → Expr → Expr
 | letE    :  Name → ...
 | lit     :  Literal
 | mdata   :  MData → Expr
 | proj    :  Name → Nat → Expr
```



Global Search: What We Do To Mathlib

```
theorem Nat.add_comm : ∀ (n m : Nat), n + m = m + n
theorem Nat.add_comm : ∀ (n m : Nat), Eq Nat (Nat.add n m) (Nat.add m n)
theorem Nat.add_comm : .mkAppN (.const `Eq) #[.const `Nat, .mkAppN (.const
`Nat.add) #[.fvar `n, .fvar `m], .mkAppN (.const `Nat.add) #[.fvar `m, .fvar
`n]]
```

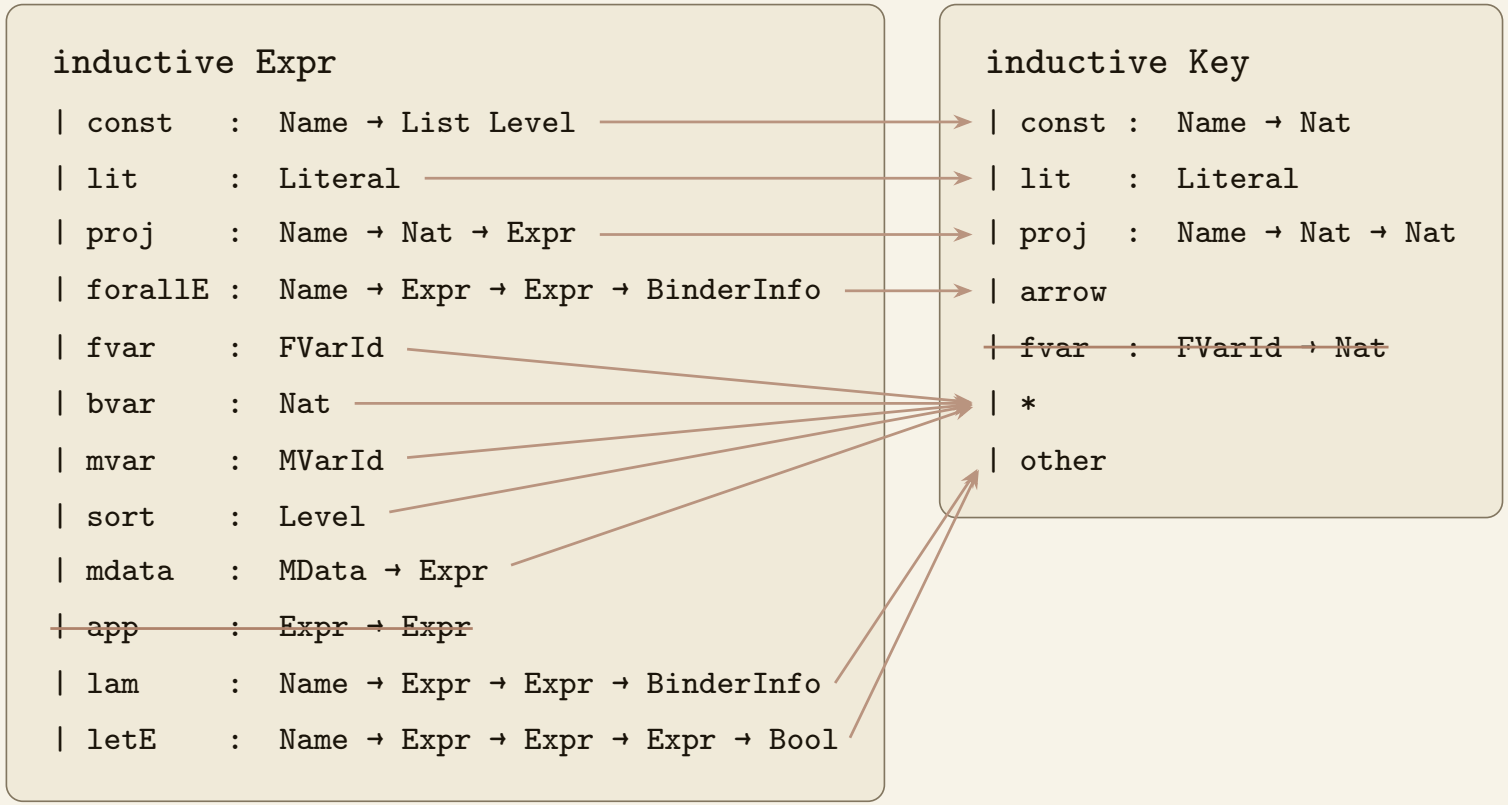
```
inductive Expr
 | bvar    :  Nat
 | fvar    :  FVarId
 | mvar    :  MVarId
 | sort    :  Level
 | const   :  Name → List Level
 | app     :  Expr → Expr
 | lam     :  Name → Expr → Expr
 | forallE :  Name → Expr → Expr
 | letE    :  Name → ...
 | lit     :  Literal
 | mdata   :  MData → Expr
 | proj    :  Name → Nat → Expr
```




```
theorem Nat.add_comm : [.const `Eq 3, .const `Nat 0, .const `Nat.add 2, .fvar
`n 0, .fvar `m 0, .const `Nat.add 2, .fvar `m 0, .fvar `n 0]
```

Global Search: What We Do To Mathlib

```
theorem Nat.add_comm : [Expr.const `Eq 3, Expr.const `Nat 0, Expr.const
`Nat.add 2, Expr.fvar `n 0, Expr.fvar `m 0, Expr.const `Nat.add 2, Expr.fvar
`m 0, Expr.fvar `n 0]
```




```
theorem Nat.add_comm : [Key.const `Eq 3, Key.const `Nat 0, Key.const `Nat.add 2,
*, *, Key.const `Nat.add 2, *, *]
```

Global Search: What We Do To Mathlib

BEFORE:

```
theorem Nat.add_comm : ∀ (n m : Nat), n + m = m + n
```

STEPS:

1. We have a Mathlib theorem

   ```
   theorem Nat.add_comm (n m : Nat) : n + m = m + n
   ```

2. Reduce the expression, strip foralls

   ```
   theorem Nat.add_comm : ∀ (n m : Nat), Eq Nat (Nat.add n m) (Nat.add m n)
   ```

3. Turn Expr tree into a list of Exprs

   ```
   theorem Nat.add_comm : [Expr.const `Eq 3, Expr.const `Nat 0, Expr.const
   `Nat.add 2, Expr.fvar `n 0, Expr.fvar `m 0, Expr.const `Nat.add 2,
   Expr.fvar `m 0, Expr.fvar `n 0]
   ```

4. Map a list of Exprs to a list of Keys

   ```
   theorem Nat.add_comm : [Key.const `Eq 3, Key.const `Nat 0,
   Key.const `Nat.add 2, *, *, Key.const `Nat.add 2, *, * ]
   ```

AFTER:

```
theorem Nat.add_comm : [Key.const `Eq 3, Key.const `Nat 0, Key.const `Nat.add 2,
*, *, Key.const `Nat.add 2, *, * ]
```

Global Search: What We Do To Our Goal

BEFORE:

```
⊢ 0 ≤ 5
```

STEPS:

1. We have a goal

   ```
   ⊢ 0 ≤ 5
   ```

2. Reduce the expression, strips foralls

   ```
   ⊢ Nat.le 0 5
   ```

3. Turn Expr tree into a list of Exprs

   ```
   ⊢ [Expr.const `Nat.le 2, Expr.lit 0, Expr.lit 5]
   ```

4. Map a list of Exprs to a list of Keys

   ```
   ⊢ [Key.const `Nat.le 2, Key.lit 0, Key.lit 5]
   ```

AFTER:

```
⊢ [Key.const `Nat.le 2, Key.lit 0, Key.lit 5]
```

Global search: DiscrTree

All Mathlib lemmas:

```
theorem Nat.add_comm :  [Key.const `Eq 3, Key.const `Nat 0, Key.const `Nat.add 2,
                           *, *, Key.const `Nat.add 2, *, *]
theorem Nat.zero_le  :  [Key.const `Nat.le 2, Key.lit 0, *]
theorem Nat.le_refl  :  [Key.const `Nat.le 2, *, *]
theorem Nat.le_succ  :  [Key.const `Nat.le 2, *, *]
```

Our goal:

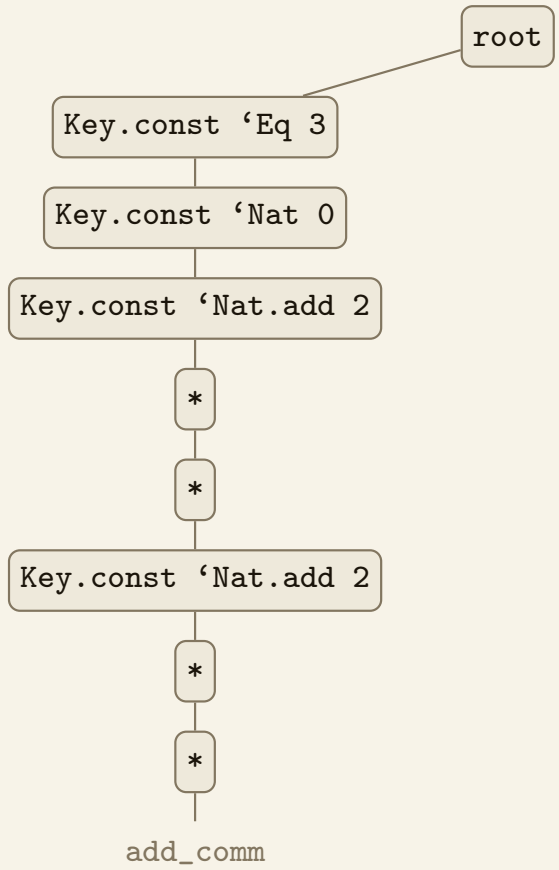
```
⊢ [Key.const `Nat.le 2, Key.lit 0, Key.lit 5]
```

Global Search: DiscrTree

```
theorem Nat.add_comm :

theorem Nat.zero_le  :

theorem Nat.le_refl  :

theorem Nat.le_succ  :
```

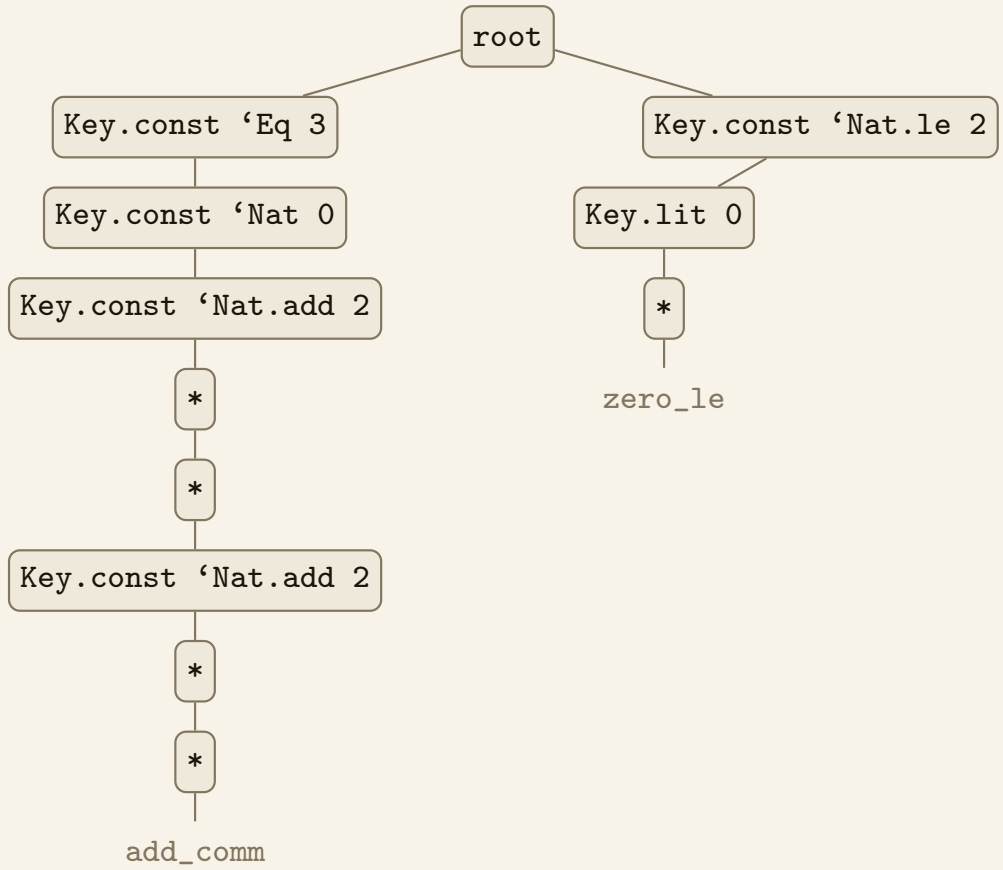
Global Search: DiscrTree

```
theorem Nat.add_comm :  [Key.const `Eq 3, Key.const `Nat 0, Key.const `Nat.add 2,
                           *, *, Key.const `Nat.add 2, *, *]
theorem Nat.zero_le  :
theorem Nat.le_refl  :
theorem Nat.le_succ  :
```



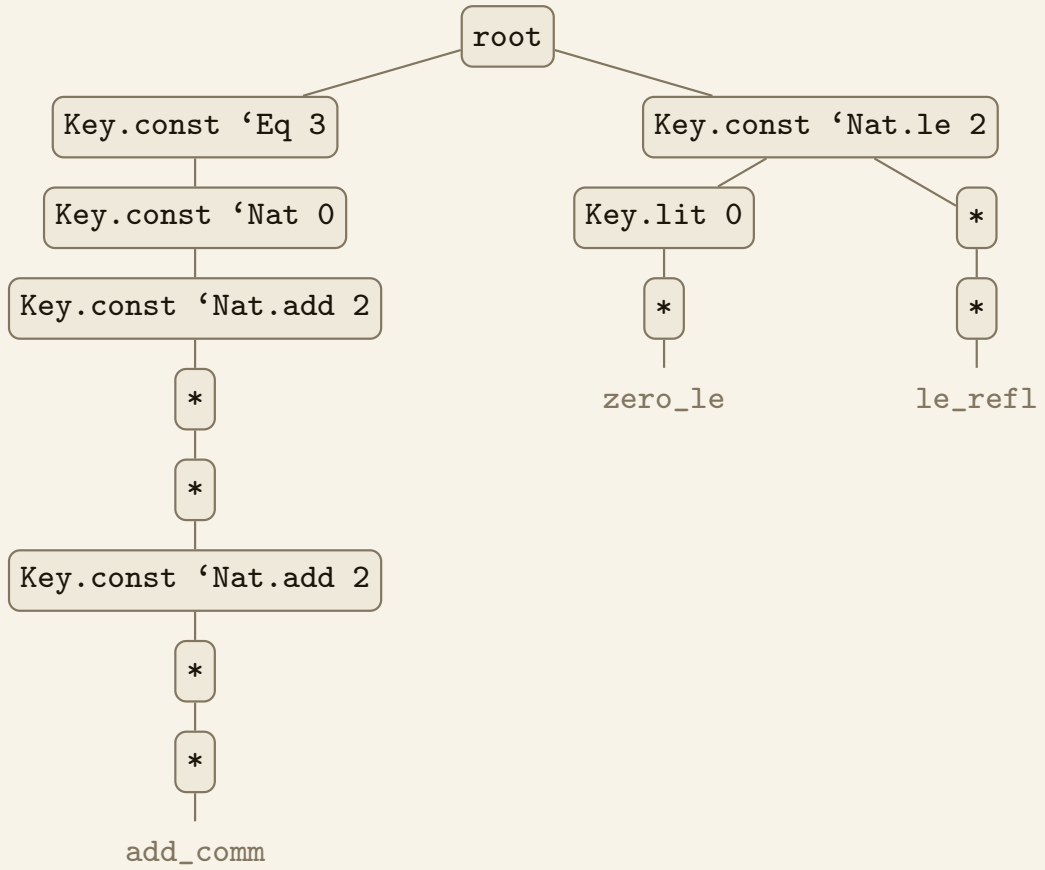
Global Search: DiscrTree

```
theorem Nat.add_comm :  [Key.const `Eq 3, Key.const `Nat 0, Key.const `Nat.add 2,
                           *, *, Key.const `Nat.add 2, *, *]
theorem Nat.zero_le  :  [Key.const `Nat.le 2, Key.lit 0, *]
theorem Nat.le_refl  :
theorem Nat.le_succ  :
```



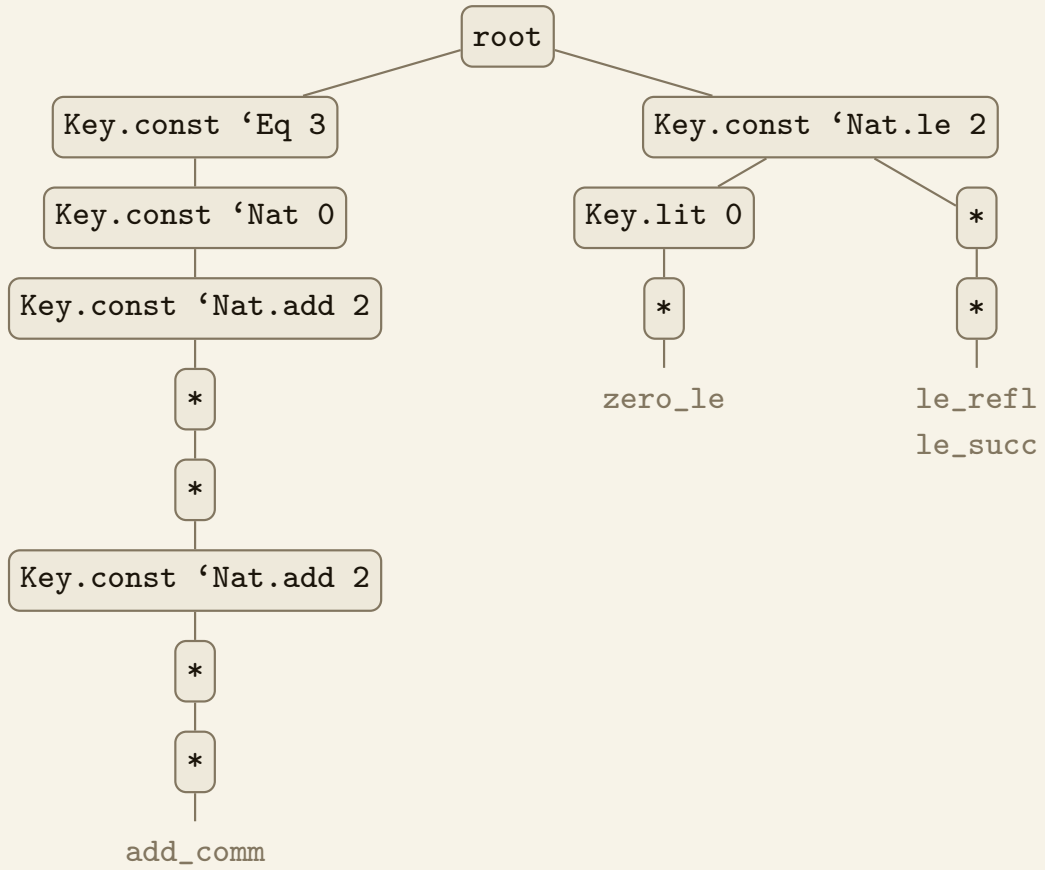
Global Search: DiscrTree

```
theorem Nat.add_comm :  [Key.const `Eq 3, Key.const `Nat 0, Key.const `Nat.add 2,
                           *, *, Key.const `Nat.add 2, *, *]
theorem Nat.zero_le  :  [Key.const `Nat.le 2, Key.lit 0, *]
theorem Nat.le_refl  :  [Key.const `Nat.le 2, *, *]
theorem Nat.le_succ  :
```



Global Search: DiscrTree

```
theorem Nat.add_comm : [Key.const `Eq 3, Key.const `Nat 0, Key.const `Nat.add 2,
                          *, *, Key.const `Nat.add 2, *, *]
theorem Nat.zero_le  : [Key.const `Nat.le 2, Key.lit 0, *]
theorem Nat.le_refl  : [Key.const `Nat.le 2, *, *]
theorem Nat.le_succ  : [Key.const `Nat.le 2, *, *]
```



Global Search: Mathlib + Our Goal

All Mathlib lemmas:

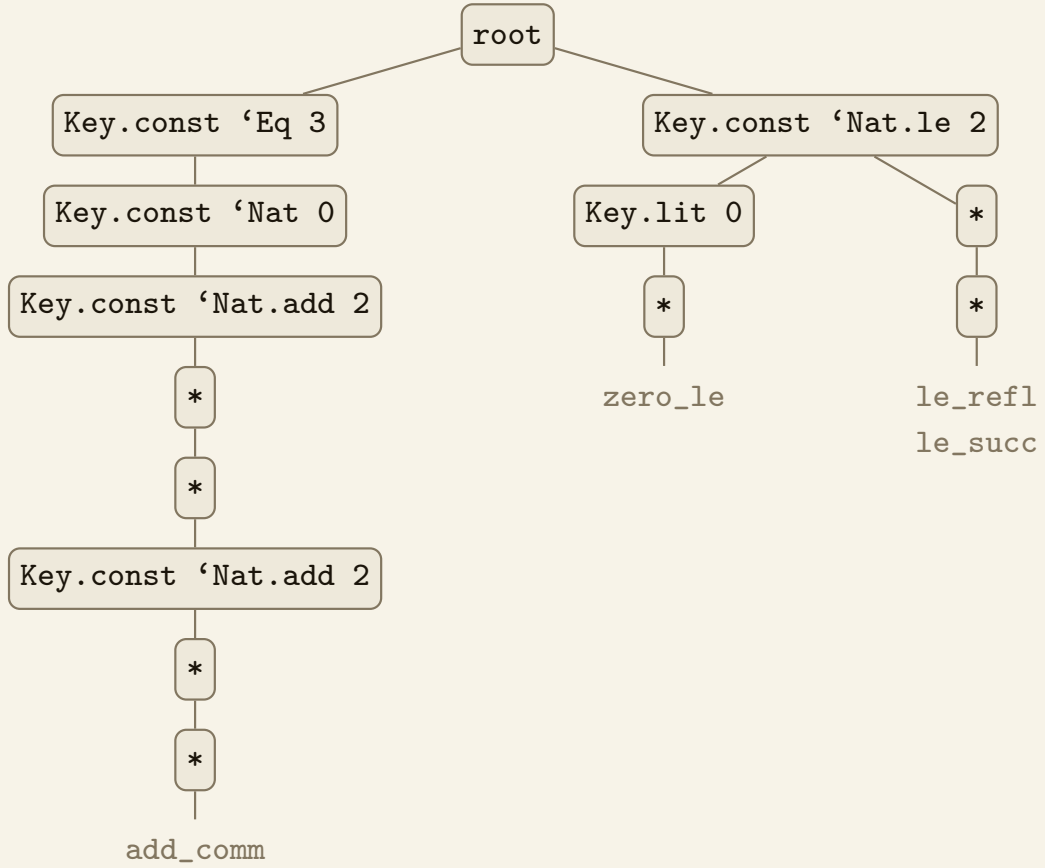
```
theorem Nat.add_comm : [Key.const `Eq 3, Key.const `Nat 0, Key.const `Nat.add 2,
                          *, *, Key.const `Nat.add 2, *, *]
theorem Nat.zero_le  : [Key.const `Nat.le 2, Key.lit 0, *]
theorem Nat.le_refl  : [Key.const `Nat.le 2, *, *]
theorem Nat.le_succ  : [Key.const `Nat.le 2, *, *]
```

Our goal:

```
⊢ [Key.const `Nat.le 2, Key.lit 0, Key.lit 5]
```

Global Search: Mathlib + Our Goal

All Mathlib lemmas:

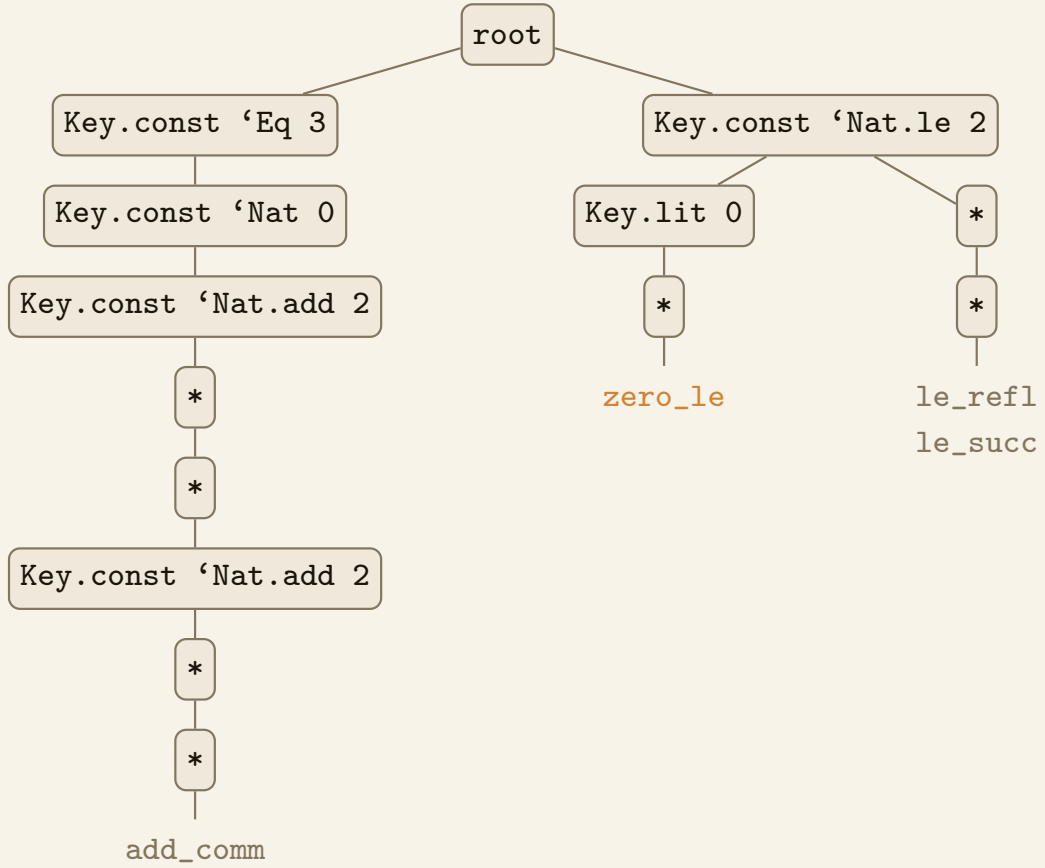


Our goal:

```
⊢ [Key.const ‘Nat.le 2, Key.lit 0, Key.lit 5]
```

Global Search: Mathlib + Our Goal

All Mathlib lemmas:

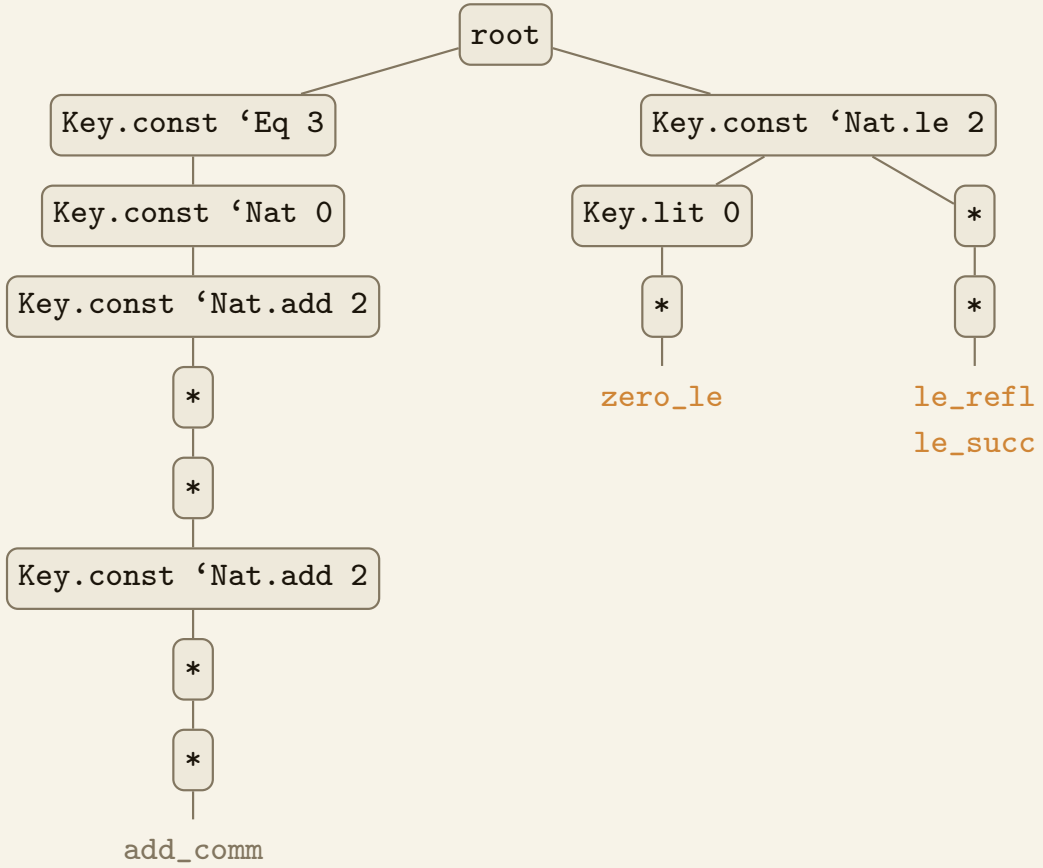


Our goal:

```
⊢ [Key.const ‘Nat.le 2, Key.lit 0, Key.lit 5]
```

Global Search: Mathlib + Our Goal

All Mathlib lemmas:

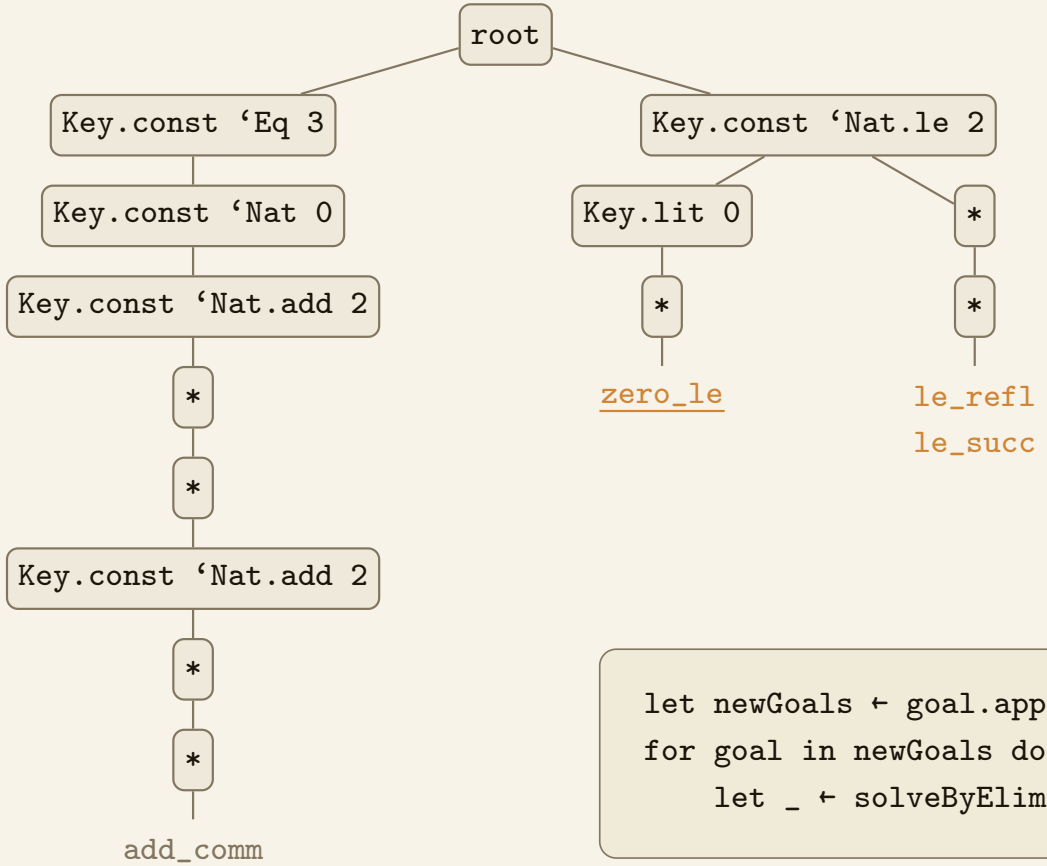


Our goal:

```
⊢ [Key.const ‘Nat.le 2, Key.lit 0, Key.lit 5]
```

Global Search: Mathlib + Our Goal

All Mathlib lemmas:



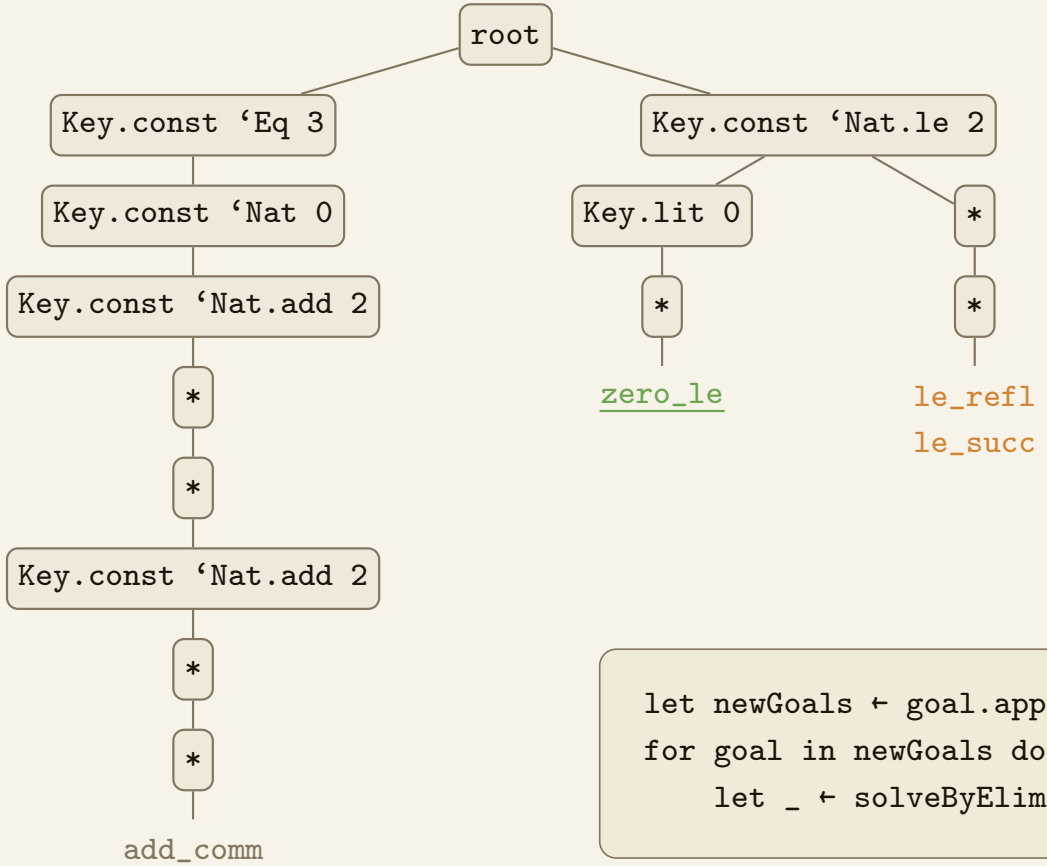

```
let newGoals ← goal.apply zero_le
for goal in newGoals do
    let _ ← solveByElim goal
```

Our goal:

```
⊢ [Key.const `Nat.le 2, Key.lit 0, Key.lit 5]
```

Global Search: Mathlib + Our Goal

All Mathlib lemmas:




```
let newGoals ← goal.apply zero_le
for goal in newGoals do
    let _ ← solveByElim goal
```

Our goal:

```
⊢ [Key.const `Nat.le 2, Key.lit 0, Key.lit 5]
```

Overview

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ :
l₃ = l₂) : l₁.length ≤ l₃.length := by
    exact?
```

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ :
l₃ = l₂) : l₁.length ≤ l₃.length := by
    exact le_of_le_of_eq h₁ (congrArg List.length (id (Eq.symm h₂)))
```

```
exact le_of_le_of_eq h₁ (congrArg List.length (id (Eq.symm h₂)))
```

Global Search (DiscrTree) — `le_of_le_of_eq`

Local Search (SolveByElim) — `h₁`

Local Search (SolveByElim) — `(congrArg List.length (id (Eq.symm h₂)))`

Overview

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ :
l₃ = l₂) : l₁.length ≤ l₃.length := by
    exact?
```

```
example (l₁ l₂ l₃ : List Nat) (h₁ : l₁.length ≤ l₂.length) (h₂ :
l₃ = l₂) : l₁.length ≤ l₃.length := by
    exact le_of_le_of_eq h₁ (congrArg List.length (id (Eq.symm h₂)))
```

```
exact le_of_le_of_eq   h₁   (congrArg List.length (id (Eq.symm h₂)))
```

Global Search (DiscrTree) — `le_of_le_of_eq`

Local Search (SolveByElim) — `h₁`

Local Search (SolveByElim) — `(congrArg List.length (id (Eq.symm h₂)))`

Demo: Real Project

Related Tactics

Related Tactics: Apply?

`apply?` and `exact?` call the same function (*/Lean/Elab/Tactic/LibrarySearch.lean*)

- `exact?` calls `exact?  ref config required requireClose := True`
- `apply?` calls `exact?  ref config required requireClose := False`

```
let newGoals ← goal.apply lemma_that_matched_our_goal_in_DiscrTree
-- lemma is guaranteed to be returned whatever happens to the remaining subgoals
for goal in newGoals do
    let _ ← solveByElim goal
```

Related Tactics: Rw?

- Builds the DiscrTree **only** from theorems of the form `x = y` or `x ↔ y`
- Puts **both sides** into the DiscrTree:

  `x`'s `[Key, Key, Key]` → DiscrTree (for `rw [lemma]`)

  `y`'s `[Key, Key, Key]` → DiscrTree (for `rw [← lemma]`)
- Subdivides our goal into **all subexpressions**,
  and matches all of them to the DiscrTree one by one

Related Tactics: Rw??

A **vscode widget** by Jovan Gerbscheid.

You write `rw??`, then **shift-click** any subexpression in the goal. The widget shows all rewrites for that exact expression.

Uses a **RefinedDiscrTree**.

Other Proof Assistants

	Search Engine	exact?	rw?
Lean	`#loogle` (command)	`exact?` / `apply?`	`rw?` / `rw??`
Rocq	`Search` (command)	`auto` (production tactic!) ▸ similar to our `solve_by_elim`, except does consider nonlocal lemmas ▸ we specify a "hint db" (doesn't index whole library) ▸ backtracking with depth 5	–
Isabelle	`find_theorems` (command)	`solve_direct` ▸ similar to our implementation ▸ but uses linear search (even though isabelle does have discr trees elsewhere) ▸ and uses the equivalent of our `assumption` to discharge the goals	–

History

pre-2018 — **Lean 2**

First *exact?* implementation (Kim Morrison) — **Mar 2019** — **Mathlib 3:** `library_search` `suggest`
Implementation: linear scan over environment

Port to Mathlib 4; Start using DiscrTree (Gabriel Ebner) — **Oct 2021** — **Mathlib 4:** `library_search`
Implementation: DiscrTree

First *rw?* implementation; Some renames (Kim Morrison) — **Jun 2023** — **Mathlib 4:** `exact?` `apply?` `rw?`
Implementation: DiscrTree
"26MB of cache to download"

Move to core; Start using LazyDiscrTree (Joe Hendrix) — **Apr 2024** — **Lean 4.7.0:** `exact?` `apply?` `rw?`
Implementation: LazyDiscrTree
"0MB of cache to download, but longer wait time for exact calls"

First *rw??* implementation (Jovan Gerbscheid) — **Jun 2025** — **Mathlib 4:** `rw??`
Implementation: RefinedDiscrTree, used as a widget

Questions